

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ»**

Факультет физико-математических и естественных наук

Кафедра математического моделирования и искусственного интеллекта

«Допустить к защите»

Заведующий кафедрой
математического моделирования
И искусственного интеллекта
д.ф.-м.н., доцент
_____ М.Д. Малых
«___» _____ 20__ г.

**Выпускная квалификационная работа
бакалавра**

Направление 09.03.03 «Прикладная информатика»

ТЕМА «Анализ эффективности ресторанного бизнеса в многомерных информационных
системах с использованием метода машинного обучения»

Выполнил студент Еюбоглу Тимур
(Фамилия, имя, отчество)

Группа НПИбд-01-22

Студ. билет № 1032224357

Руководитель выпускной
квалификационной работы
Фомин М. Б. к.ф.-м.н., доц.
(Ф.И.О., степень, звание, должность)

(Подпись)

Автор _____
(Подпись)

г. Москва

2026 г.

Электронная копия документа



**Федеральное государственное автономное образовательное учреждение
высшего образования
«Российский университет дружбы народов имени Патриса Лумумбы»**

**АННОТАЦИЯ
выпускной квалификационной работы**

Еюбоглу Тимур

(фамилия, имя, отчество)

на тему:

Выпускная квалификационная работа посвящена разработке аналитической системы для ресторанного бизнеса на основе OLAP-хранилища данных, интерактивной визуализации и методов машинного обучения. В центре исследования находится задача преобразования разрозненных сведений о продажах, клиентах, ресторанах и меню в единую среду, которая помогает оценивать финансовые показатели, выявлять закономерности спроса и строить прогнозы.

Работа включает три основных раздела. В первом разделе рассмотрены принципы многомерных информационных систем, архитектура распределённых хранилищ данных, подходы к размещению кубоидов и выбору оптимального пути выполнения OLAP-запросов. Во втором разделе проанализирована предметная область ресторанной аналитики, выполнено сравнение СУБД и средств доступа к базе данных из Python, а также построена модель хранилища по схеме «звезда». В третьем разделе описана реализация прототипа: создано локальное аналитическое хранилище, разработан веб-дашборд для многомерного анализа, подготовлены отчёты по ключевым показателям и встроен модуль прогнозирования продаж.

Автор ВКР

(Подпись)

(ФИО)

Оглавление

Введение	4
1. Обзор концепции многомерных информационных систем.....	6
1.1. Архитектура распределенного хранилища данных	6
1.2. Достижения в сфере распределённых систем хранения данных	9
1.3. Модели на основе кубоидов.....	10
2. Анализ предметной области.....	19
2.1. Обзор и сравнение СУБД как источника данных	22
2.2. Обзор и сравнение средств доступа к СУБД из Python	25
2.3. Моделирование базы данных.....	29
3. Реализация прототипа аналитической системы	36
3.1. Реализация хранилища данных.....	36
3.2. Примеры аналитических отчетов	40
3.3. Использование машинного обучения в анализе данных	48
Заключение	53
Список источников.....	54

Введение

Современный ресторанный бизнес ежедневно формирует большой объём данных: заказы, чеки, позиции меню, сведения о клиентах, скидках, времени обслуживания, выручке и прибыли. Сами по себе эти записи не дают руководителю готового понимания ситуации. Чтобы увидеть слабые места, оценить спрос и принять обоснованное решение, бизнесу нужен инструмент, который связывает накопленные сведения в единую аналитическую картину. В ресторанной сфере ценность аналитики особенно высока. Спрос зависит от дня недели, сезона, времени суток, района, формата заведения, клиентского сегмента и состава меню. Без системной обработки эти зависимости остаются незаметными. Поэтому разработка аналитической системы, которая объединяет хранилище данных, визуальные отчёты и машинное обучение, представляет практический интерес и отвечает потребностям бизнеса.

Актуальность работы связана с ростом роли данных в управлении предприятиями общественного питания. Ресторанная сеть должна быстро реагировать на изменение спроса, контролировать прибыльность блюд, оценивать работу филиалов, планировать закупки и распределять персонал. Если руководство опирается только на отдельные отчёты или ручной анализ таблиц, оно теряет скорость и часто видит проблему уже после её появления.

Цель работы — разработать прототип аналитической системы для ресторанного бизнеса, которая на основе OLAP-хранилища данных обеспечивает многомерный анализ показателей, визуализацию результатов и прогнозирование ключевых бизнес-метрик с применением методов машинного обучения.

Для достижения цели необходимо решить следующие задачи:

- Рассмотреть принципы построения многомерных информационных систем и распределённых хранилищ данных.
- Изучить предметную область ресторанного бизнеса и определить показатели, значимые для анализа продаж, прибыли, спроса и клиентского поведения.

- Спроектировать модель хранилища данных по схеме «звезда» с таблицей фактов и набором измерений.
- Реализовать локальное аналитическое хранилище, наполнить его демонстрационными данными и подготовить представления для отчётов.
- Разработать интерактивный дашборд для анализа ресторанных показателей.
- Подготовить модуль машинного обучения для прогноза выручки, прибыли, количества проданных блюд и числа уникальных клиентов.
- Оценить качество прогнозных моделей и определить направления дальнейшего развития системы.

Объект исследования — процессы анализа данных в ресторанном бизнесе.

Предмет исследования — методы и программные средства построения OLAP-хранилища, визуализации ресторанных показателей и прогнозирования продаж с использованием машинного обучения.

Научная новизна работы заключается в построении единого аналитического контура, который объединяет OLAP-модель ресторанных данных, интерактивный многомерный анализ и прогнозирование бизнес-показателей. В отличие от простого отчётного решения, разработанный прототип связывает структуру хранилища с машинным обучением: система использует календарные, ресторанные, клиентские и лаговые признаки для оценки будущей динамики.

В работе использованы методы системного анализа, сравнительного анализа программных средств, моделирования баз данных, OLAP-проектирования, обработки табличных данных, визуального анализа и машинного обучения. Для реализации прототипа применены Python, DuckDB, SQLAlchemy, pandas, Plotly, Dash и scikit-learn. Для оценки качества прогнозов использованы метрики MAE, RMSE, MAPE и R^2 . При проектировании хранилища использована схема «звезда», которая включает таблицу фактов продаж и измерения даты, времени, ресторана, меню и клиента.

1. Обзор концепции многомерных информационных систем

Развитие Интернета и прогресс в области информационных технологий привели к растущему спросу предприятий на доступ в режиме реального времени к широкому спектру онлайн-информации, начиная от общих аспектов ведения бизнеса и заканчивая конкретными, углубленными данными по различным темам. Этот поиск знаний привел к тому, что инструменты онлайн-анализа (OLAP) стали играть ключевую роль, позволяя предприятиям быстро принимать обоснованные решения. Традиционно инструменты OLAP работали на основе централизованного хранилища данных, где данные были тщательно организованы вокруг центральной темы, представленной таблицей фактов. Однако создание и обслуживание единого централизованного хранилища данных по своей сути является трудоемким и дорогостоящим мероприятием. Более того, это может привести к некоторой негибкости архитектурной структуры, особенно когда операционные системы охватывают широко разбросанные географические области. В таких ситуациях предприятия все чаще ищут гибкие и экономически эффективные альтернативы для удовлетворения своих сложных потребностей в данных в различных областях бизнеса. Именно здесь концепция распределенного хранилища данных становится возможным решением. В распределенном хранилище данных данные стратегически распределяются по нескольким сайтам или центрам обработки данных, каждый из которых работает независимо. Основная цель этих распределенных хранилищ данных - повысить доступность данных на локальных сайтах и одновременно сократить время обработки локальных запросов

1.1. Архитектура распределенного хранилища данных

Архитектура распределенного хранилища данных обычно состоит из отдельных компонентов, соединенных между собой через сеть и работающих сообща для создания единого глобального хранилища данных, даже если сами данные физически распределены по разным местоположениям. По сути, распределенное хранилище данных функционирует как центральный узел всех

корпоративных данных, обеспечивая беспрепятственную передачу необходимых данных в отдельные витрины данных. Впоследствии пользователи могут получить доступ к необходимой им информации для множества целей, таких как управление заказами, выставление счетов клиентам, анализ продаж, отчетность и различные аналитические функции. Используя модель распределенного хранилища данных, предприятия могут получить множество преимуществ. Эти преимущества включают повышение производительности запросов за счет доступности локализованных данных, экономичность за счет устранения сложностей, связанных с единым централизованным хранилищем данных, большую гибкость и масштабируемость для удовлетворения меняющихся требований к данным, а также повышение локальной автономии, позволяющее отдельным сайтам сохранять контроль над своими данными и выполнять локальные вычисления. Кроме того, распределение данных по нескольким узлам повышает избыточность и обеспечивает доступность данных даже в случае системных сбоев. В целом, модель распределенного хранилища данных эффективно решает задачи, связанные с централизованными хранилищами данных, предоставляя предприятиям более гибкое решение для быстрого и точного доступа к данным для удовлетворения широкого спектра бизнес-потребностей. В последние годы были предложены и внедрены различные архитектуры распределенных хранилищ данных для решения проблем, связанных с централизованными хранилищами данных (Albrecht & Lehner). В таблице 1 приведены краткие сведения о некоторых из этих моделей

Таблица 1.1 Подходы к организации хранилища данных в литературе

Модель	Описание	Преимущества	Проблемы / недостатки
Распределённые корпоративные хранилища данных (Albrecht & Lehner,)	Хранилище данных распределено между несколькими площадками или дата-центрами, каждая из которых обслуживает разные бизнес-подразделения или отделы.	Локальная автономия, снижение нагрузки на центральную систему	Сложности интеграции данных. Умеренное время отклика запросов
Подход Инмона (Inmon's Approach)	Акцент на нормализации данных и централизованном репозитории. Данные интегрируются и преобразуются перед загрузкой.	Согласованность данных, единая версия истины	Проблемы производительности при больших объёмах данных. Увеличенное время ответа для сложных запросов. Высокая нагрузка на центральное хранилище
Подход Уайта (White's Approach)	Ориентирован на создание витрин данных (data marts) в разных узлах, адаптированных под конкретные аналитические задачи. Формируются на основе центральных или исходных данных.	Гибкость, адаптация под локальные потребности	Проблемы согласованности между витринами данных. Увеличенное время выполнения OLAP-запросов в распределённой среде
Архитектура Skalla (Kashfi & Hajmoosaei)	Состоит из локальных хранилищ данных, расположенных рядом с точками сбора данных и координируемых центральным компонентом.	Эффективная обработка и распределение данных	Возможные задержки и проблемы сетевого трафика. Очень высокое время выполнения OLAP-запросов в распределённой среде

Обращаясь к таблице 1, мы видим, что во всех моделях распределенных хранилищ данных основной целью является поддержка быстрого и точного принятия решений с помощью локальных витрин данных. Однако сложности возникают, когда сложные запросы требуют взаимодействия между сайтами и переноса данных, что приводит к задержкам и увеличению сетевого трафика. Для решения этих проблем современные модели распределенных хранилищ данных предполагают ведение таблицы фактов на централизованном сайте. Хотя этот подход снижает затраты на передачу данных по сети для сложных OLAP-запросов (Keshin & Yazıcı, 2023), которые требуют межсайтовой связи, он также снижает автономность принятия решений на местах на участвующих сайтах, потенциально перегружая централизованное хранилище данных. Достижение баланса между

централизованным управлением и локальной автономией является ключевой задачей при эффективном внедрении распределенных хранилищ данных

1.2. Достижения в сфере распределённых систем хранения данных

В сфере распределённых систем хранения данных (Distributed Data Warehousing Systems, DDWS) были разработаны различные методы, повышающие их эффективность и применимость в различных областях. К таким методам относятся создание специализированных витрин данных для конкретных сфер бизнеса (Shaweta, 2014), выбор между централизованными хранилищами данных и унифицированными концептуальными схемами (Link, Schewe, & Zhao, 2006; Zaho & Schewe, 2004), а также использование хранилищ информационных данных для централизации и предоставления лицам, принимающим решения, доступа к точным данным (Yeruva & Kumar, 2010). Усилия по внедрению операционных баз данных для запросов в рамках оперативной аналитической обработки (Online Analytical Processing, OLAP) позволили улучшить аналитику в режиме реального времени (Akinde, Böhlen, Johnson, Lakshmanan, & Srivastava, 2003), а абстрактные конечные автоматы (Abstract State Machines, ASM) оптимизируют структуру хранилищ данных (Link et al., 2006). Архитектура Data Mining Grid (DMGA) упрощает процессы интеллектуального анализа данных (Pérez, Sánchez, Robles, Herrero, & Peña, 2007), а многоагентные системы повышают эффективность обработки запросов (Rao, 2009).

Кроме того, внедрение метапоисковой системы для поиска научных данных позволяет уделять больше внимания разработке схем данных (Viloria, Rodado, & Lezama, 2019). Реляционная распределённая система хранения данных на кластерах без совместного использования ресурсов повышает масштабируемость и производительность (Choi et al., 2013), а фреймворк на основе онтологий улучшает интеграцию источников данных (Browne, O'Reilly, Hutchinson, & Krdzavac, 2019). В недавних исследованиях изучались распределённые фреймворки для обработки и анализа потоков данных (Isah et al., 2019), а распределённая сервис-

ориентированная архитектура для интернета вещей позволяет адаптировать сервисы и сбор данных на периферии (Zhu, Dhe-lim, Zhou, Yang, & Ning, 2019). Трехуровневое хранилище медицинских данных обеспечивает конфиденциальность и безопасность данных (Шахид, Нгуен, Кечади, 2021), а сочетание Hive, MongoDB и Cassandra в хранилище сельскохозяйственных данных позволяет эффективно анализировать данные (Нго, Ле-Кхак, Кечади, 2019). Адаптивный подход к работе в режиме реального времени решает проблемы интеграции финансовых данных (Фикри, Рида, Абгур, Муссаид, Эль-Омри, 2019), а архитектура распределенной фабрики данных с графом знаний на основе метаданных повышает эффективность управления данными (Ли, Ян, Ся, Чжан, Лю, 2022). Наконец, целостный подход к архитектуре хранилищ данных подчеркивает важность конфиденциальности и безопасности в сфере здравоохранения (Танталайдж, Ле-Кхак, Кечади, 2023). Эти продолжающиеся разработки формируют будущее управления данными, позволяя организациям в полной мере использовать свои информационные ресурсы.

1.3. Модели на основе кубоидов

Предложенная модель распределения кубоидов от центрального узла базового кубоида ко всем участвующим узлам в среде распределенного хранилища данных может быть реализована тремя способами: (i) распределение кубоидов на основе балансировки нагрузки, (ii) распределение кубоидов на основе максимального количества локальных запросов с каждого узла и (iii) формирование кубоидов после разделения данных по значению параметра «Местоположение

Алгоритм распределения кубоидов на основе балансировки нагрузки

Из n участвующих в таблице фактов измерений на узле k (центральном узле), где хранится таблица фактов, формируется $2n$ кубоидов. Каждый из этих кубоидов будет распределен между другими узлами, отличными от узла k , в зависимости от прогнозируемой нагрузки (Кастаньоли, 1986) и частоты запросов, поступающих с каждого узла. Среди соседних узлов узел i ($i \neq (k)$) с минимальной нагрузкой

определяется по формуле $B = \min(w_1 \times (l_1 + d_1), \dots, w_n \times (l_n + d_n))$. После определения узла/сайта i с минимальным значением общей нагрузки (в процентах от загрузки процессора) ему выделяется кубоид с наибольшим тековым значением. Это распределение кубоидов является итеративным процессом. Поэтому после выделения кубоида с наибольшим тековым значением снова вычисляются значения общей нагрузки для каждого соседнего узла k . Алгоритм 1 описывает алгоритм распределения кубоидов на основе балансировки нагрузки.

Алгоритм 1: Балансировщик кубоидов (CuboidLoadBalancer, CLB)

Вход: Список шаблонов кубоидов: $C = \{C_1, C_2, \dots, C_n\}$

Выход: Подходящие узлы (сайты) для размещения кубоидов

Процедура: Вычислить значение тега для каждого шаблона кубоида: $TV(C_i)$, где $i = 1 \dots n$. Сохранить значения тегов в списке (LIST) в порядке убывания.

Псевдокод:

```

1: пока LIST не пуст do
2:   TV_k ← извлечь первое значение тега из LIST
3:   min_load ← ∞
4:   для каждого соседнего узла i для k do
5:     вычислить функцию общей нагрузки: TL(i, TV_k)
6:     если TL(i, TV_k) < min_load тогда
7:       min_load ← TL(i, TV_k)
8:       B ← i
9:   конец если
10:  конец цикла
11: конец цикла
12: разместить кубоид, соответствующий TV_k, на узле B

```

Конец алгоритма

Временная сложность алгоритма распределения кубоидов на основе балансировки нагрузки может быть проанализирована с учетом количества необходимых итераций и вычислительной сложности операций, выполняемых на каждой итерации.

Рассмотрим временную сложность каждого этапа.

Шаг 1. Для вычисления значения тега для каждого кубоида необходимо перебрать все кубоиды и выполнить необходимые вычисления. Временная сложность этого этапа обычно составляет $O(n)$, где n — количество кубоидов.

Шаг 2. Цикл на шаге 2 выполняется до тех пор, пока список не опустеет.

Количество итераций зависит от количества кубоидов и распределения рабочей нагрузки. В худшем случае, когда каждый кубоид нужно распределить между разными площадками, количество итераций составит $O(n)$.

На каждой итерации шага 2 вычисления, выполняемые на шагах 2.1 и 2.2, в целом можно считать выполняемыми с постоянной временной сложностью, поскольку они представляют собой базовые операции с данными.

На шаге 2.3 кубоид распределяется между выбранными площадками, что может потребовать обновления данных или выполнения других связанных операций. Временная сложность этого шага зависит от конкретных особенностей реализации и сложности процесса распределения.

Она может варьироваться от $O(1)$ при простых обновлениях до более сложных временных сложностей в зависимости от используемых алгоритмов и структур данных.

На шаге 2.4 необходимо обновить информационную матрицу кубоидов в соответствии с расположением кубоида. Временная сложность этого шага также зависит от особенностей реализации, таких как структура данных, используемая для матрицы, и сложность процесса обновления. Как и в случае с шагом 2.3, она может варьироваться от $O(1)$ до более сложных временных сложностей.

В целом, если рассматривать наихудший сценарий, временную сложность алгоритма можно оценить как $O(n)$, где n — количество кубоидов. Однако важно отметить, что фактическая временная сложность может варьироваться в зависимости от конкретной реализации.

Алгоритм распределения кубоидов, основанный на максимизации количества локальных запросов с каждого узла

Для схемы распределения определяется коэффициент полезного действия, и каждый кубоид распределяется по конкретному узлу (без репликации).

Распределение кубоидов по конкретным узлам осуществляется следующим образом. Коэффициент полезного действия при распределении конкретного кубоида i по конкретному узлу j определяется следующим образом.

$$B_{ij} = \sum_k f_{kj} \times \sum_i D_{il} \times q_{ki}$$

B_{ij} — коэффициент полезного действия при распределении конкретного набора атрибутов i по конкретному узлу j , i — индекс кубоида, j — индекс узла, k — индекс OLAP-запроса, f_{kj} — частота выполнения запроса k на узле j . D_{il} — измерение, присутствующее в кубоиде D_i , к которому обращается запрос k . Конкретный кубоид i распределяется по узлу с максимальным значением коэффициента полезного действия.

Этот метод формализован в виде алгоритма 2 для распределения кубоидов по разным узлам.

Алгоритм 2: Распределитель кубоидов (CuboidAllocator, CA)

Вход:

- n — количество кубоидов
- m — количество узлов (сайтов)
- $F[n][m]$ — двумерный массив, представляющий частоту запросов для каждого кубоида на каждом узле
- $D[n][m][k]$ — трёхмерный массив, представляющий измерения, используемые каждым кубоидом и запросом на каждом узле
- $Q[n][m][k]$ — трёхмерный массив, представляющий коэффициент влияния каждого запроса на каждый кубоид на каждом узле

Выход: $site_allocation[n]$ — массив, хранящий индекс узла, на который назначен каждый кубоид

Шаг 1: Вычисление меры выгоды (Benefit Measure)

```

1: для  $i$  от 1 до  $n$  do
2:   для  $j$  от 1 до  $m$  do
3:      $F\_sum \leftarrow 0$ 
4:     для  $k$  от 1 до  $number\_of\_queries$  do
5:        $F\_sum \leftarrow F\_sum + F[i][j]$    ▷ суммарная частота запросов
6:     конец цикла

7:    $D\_sum \leftarrow 0$ 
8:   для  $k$  от 1 до  $number\_of\_queries$  do
9:      $D\_sum \leftarrow D\_sum + D[i][j][k]$    ▷ сумма используемых измерений
10:  конец цикла

```

```

11:      Q_sum ← 0
12:      для k от 1 до number_of_queries do
13:          Q_sum ← Q_sum + Q[i][j][k]    ▷ общее влияние запросов
14:      конец цикла

15:      B[i][j] ← F_sum × D_sum × Q_sum    ▷ вычисление меры выгоды
16:  конец цикла
17: конец цикла

```

Шаг 2: Назначение кубоидов на узлы

```

1: для i от 1 до n do
2:     max_benefit ← -1
3:     allocated_site ← -1

4:     для j от 1 до m do
5:         если B[i][j] > max_benefit тогда
6:             max_benefit ← B[i][j]
7:             allocated_site ← j
8:         конец если

9:     site_allocation[i] ← allocated_site    ▷ сохранить выбранный узел
10:  конец цикла
11: конец цикла

12: вернуть site_allocation

```

Конец алгоритма

Такой подход приводит к увеличению количества локальных запросов, хотя экспериментальные результаты показывают, что при использовании этой стратегии распределение нагрузки может быть неравномерным.

Алгоритм формирования кубоидов после разбиения данных по измерению

В основе этого подхода лежит проблема, присущая централизованному хранилищу данных, то есть отсутствие возможностей локальной обработки и анализа. Как правило, "Местоположение" является одним из наиболее важных аспектов бизнес-анализа. Это знание побудило нас предложить данный подход, основанный на методе фрагментации (ramachandran, Nair, & Jasmi).

Метод фрагментации базы данных в основном подразделяется на два метода: (i) горизонтальная фрагментация и (ii) Вертикальная фрагментация. При горизонтальной фрагментации схема фрагментов остается такой же, как и в исходном отношении, тогда как при вертикальной фрагментации схемы фрагментов отличаются от исходного отношения.

В предлагаемом нами методе мы использовали метод горизонтальной фрагментации, при котором фрагменты создаются на основе некоторой выборки,

известной как предикаты (Ramachandran et al.). В отношении (измерениях в таблице фактов) может быть много атрибутов, по которым могут быть построены предикаты.

Алгоритм 3: Горизонтальное фрагментирование (Horizontal Fragmentation, HF)

Вход:

- `fact_table` — исходная таблица фактов, содержащая данные для всех локаций
- `m` — количество узлов (сайтов)
- `locations[m]` — массив, содержащий названия локаций для каждого узла

Выход:

- `fragmented_tables[m]` — массив фрагментированных таблиц фактов (по одной для каждого узла)

Шаг 1: Инициализация переменных

```
1: fragmented_tables[m]   ▷ создать пустой массив для хранения фрагментов таблицы фактов
```

Шаг 2: Горизонтальное фрагментирование по локациям

```
1: для i от 1 до m do
2:   location_name ← locations[i]   ▷ получить название локации для текущего узла

3:   fragment_i ← EmptyTable()   ▷ создать пустую таблицу для текущего фрагмента

4:   для каждой строки row в fact_table do
5:     если row.Location == location_name тогда
6:       fragment_i.addRow(row)   ▷ добавить строку в фрагмент
7:     конец если
8:   конец цикла

9:   fragmented_tables[i] ← fragment_i   ▷ сохранить фрагмент в массиве
10: конец для
```

Шаг 3: Результат

```
вернуть fragmented_tables
```

Результатом является массив таблиц, где каждая таблица содержит только те строки, которые относятся к конкретной локации (узлу).

После горизонтального разбиения таблицы фактов на фрагменты меньшего размера, основанные на местоположении различных сайтов, они переносятся и сохраняются на соответствующих сайтах. Каждый из этих фрагментов теперь будет действовать как таблица фактов (базовый кубоид) для отдельных сайтов.

Алгоритм выбора оптимального пути выполнения olap-запроса

Предложен алгоритм для оптимального плана выполнения OLAP-запроса в Cuboids в распределенной среде, который формализован в виде алгоритма-4 под названием OLAPQueryPathOptimizer (OQPO).

Вот перевод псевдокода на русский язык с сохранением структуры и обозначений:

Алгоритм 4: Оптимизатор пути OLAP-запроса (OLAPQueryPathOptimizer, OQPO)

Вход:

- Q — агрегирующий запрос, представляющий целевой шаблон
- L — местоположение (узел) целевого шаблона
- $|D_i|$ — кардинальность каждой размерности базового кубоида или таблицы фактов
- S_i — размер каждой размерности
- T_G — стоимость вычислений (операции Roll-up)
- T_C — стоимость сетевого взаимодействия

Выход:

- C — множество промежуточных кубоидов, задающих оптимальный путь от начального шаблона к целевому

Псевдокод:

```
1: функция OQPO(Q, L,  $|D_i|$ ,  $S_i$ , T_G, T_C)
2:   C ← пустое множество

3:   Q_i ← GenerateQuery(Q)
4:   TP ← IdentifyDimensions(Q_i)
5:   TV ← CalculateTaggedValue(TP,  $|D_i|$ ,  $S_i$ )

6:   C ← FindCuboidsContainingPattern(TV, L)

7:   если размер(C) > 1 тогда
8:     c* ← null
9:     min_TTOT ← ∞

10:    для каждого c в C do
11:      TTOT ← T_G(c) + T_C(c)
12:      если TTOT < min_TTOT тогда
13:        min_TTOT ← TTOT
14:        c* ← c
15:      конец если
16:    конец цикла
```



```

17:      Output  $\leftarrow$  RetrieveOutput(c*)
18:      ShipResult(Output, Q_i)
19:  конец если

20:  вернуть C
21:  конец функции

```

Псевдокод предполагает наличие определенных функций:

- **GenerateQuery(Q):** Генерирует запрос с определенного сайта.
- **IdentifyDimensions(Q_i):** Идентифицирует измерения, входящие в группу, по выражению запроса.
- **CalculateTaggedValue(TP, |D_i|, S_i):** Вычисляет значение tagged для целевого шаблона, используя размеры, количество элементов и типоразмеры.
- **FindCuboidsContainingPattern(TV, L):** Выполняет поиск кубоидов, содержащих целевой шаблон, и определяет их местоположение.
- **TG(c):** Вычисляет стоимость вычислений для выполнения операций свертывания кубоида c.
- **TC(c):** Вычисляет стоимость сетевого взаимодействия, связанную с доступом к cuboid c.
- **RetrieveOutput(c*):** Извлекает желаемый результат из выбранного cuboid c*.
- **ShipResult(вывод, Q_i):** Отправляет результат на исходный сайт.

Временная сложность алгоритма для оптимального плана выполнения OLAP-запроса может быть проанализирована на основе сложности каждого шага и количества требуемых итераций. Давайте разберем временную сложность для каждого шага: Временная сложность алгоритма для оптимального плана выполнения OLAP-запроса может быть проанализирована на основе сложности каждого шага и количества требуемых итераций. Давайте разберем временную сложность для каждого шага:

Шаг 1: Генерация запроса с любого сайта i обычно имеет постоянную временную сложность, обозначаемую как $O(1)$.

Шаг 2: Определение измерений в группе по выражению и пометка их в качестве целевого шаблона включает в себя проверку запроса и выбор соответствующих измерений. Временная сложность зависит от размера запроса и

количества задействованных измерений. В общем, можно считать, что временная сложность этого шага равна $O(|Q_i|)$, где $|Q_i|$ представляет собой размер запроса.

Шаг 3: Вычисление помеченного значения целевого шаблона включает в себя выполнение вычислений, основанных на измерениях, мощностях и размерах базового кубоида. Временная сложность зависит от количества измерений и сложности выполняемых вычислений. В общем, можно считать, что временная сложность этого шага равна $O(|D|)$, где $|D|$ представляет количество измерений.

Шаг 4: Поиск кубоидов, содержащих целевой шаблон в качестве подмножества, и определение их местоположения включает поиск по информационной матрице кубоидов. Временная сложность зависит от структуры матрицы и используемого алгоритма поиска. В общем, можно считать, что временная сложность этого шага равна $O(|CIM|)$, где $|CIM|$ представляет размер информационной матрицы в форме куба.

Шаг 5: Вычисление общего времени доступа (TTOT) для каждого кубоида и выбор того, у которого TTOT меньше, включает в себя выполнение вычислений, основанных на стоимости вычислений и стоимости сетевой связи. То временная сложность зависит от количества кубоидов и сложности выполняемых вычислений. В целом, можно считать, что временная сложность этого шага равна $O(|C|)$, где $|C|$ представляет количество кубоидов.

Шаг 6: Получение желаемого результата из выбранного кубоида обычно требует постоянной временной сложности, обозначаемой как $O(1)$.

Шаг 7: Отправка результата на сайт i обычно требует постоянной временной сложности, обозначаемой как $O(1)$.

Шаг 8: Возврат набора промежуточных кубоидов, предполагающих, что оптимальный план выполнения имеет временную сложность $O(1)$, поскольку он включает в себя возврат структуры данных.

В целом, учитывая наихудший сценарий, временную сложность алгоритма можно приблизительно определить как $O(|Q_i| + |D| + |CIM| + |C|)$, где $|Q_i|$ представляет размер запроса, $|D|$ представляет количество измерений, $|CIM|$

представляет размер информационной матрицы кубоида, а $|C|$ представляет количество кубоидов.

2. Анализ предметной области

В данном разделе определяется программный стек, на котором будет строиться система анализа данных для ресторанного бизнеса. Его подбор напрямую связан с характером решаемой задачи: необходимо не просто получить сведения из хранилища, а выстроить целостный процесс работы с ними — от выборки и подготовки до аналитической интерпретации и построения прогнозных моделей. Поэтому на этом этапе важно обозначить, какие программные средства смогут обеспечить устойчивую, понятную и технологически согласованную реализацию всех этапов обработки данных.

Разрабатываемая система ориентируется на работу с данными, которые накапливаются в OLAP-структуре и отражают ключевые показатели деятельности предприятия общественного питания. В рамках этой логики система должна извлекать сведения из хранилища, преобразовывать их в удобный для дальнейшей обработки вид, очищать выборки от пропусков и несогласованностей, а затем подготавливать массивы признаков для аналитических процедур. Такой подход позволяет связать исходную информацию из базы с последующим этапом интеллектуального анализа без разрыва между хранением и вычислительной частью.

Следующий блок задач связан уже не с получением данных, а с их исследованием. Система должна выявлять зависимости между показателями, показывать динамику продаж, фиксировать изменения спроса, помогать в оценке среднего чека, сезонных колебаний и иных характеристик, значимых для ресторанного бизнеса. Для этого необходимы инструменты визуализации, способные представить результат в наглядной форме — через графики, диаграммы и иные аналитические представления. Визуальный слой в такой архитектуре играет

не вспомогательную, а содержательную роль, поскольку именно он делает числовые зависимости интерпретируемыми.

Наряду с этим проект предполагает применение методов машинного обучения. На основе исторических данных система должна формировать модели, которые смогут решать прикладные задачи прогнозирования, классификации или сегментации. Иначе говоря, программный стек должен поддерживать не только стандартную обработку таблиц, но и обучение алгоритмов, проверку их качества, а также последующее использование полученных моделей в аналитическом контуре.

Отсюда вытекают и критерии выбора инструментов. Они должны корректно взаимодействовать с реляционной СУБД, удобно работать с крупными аналитическими выборками, поддерживать операции над табличными структурами и обеспечивать реализацию алгоритмов машинного обучения без избыточного усложнения архитектуры. Дополнительно значение имеют распространенность библиотек, качество документации, устойчивость экосистемы и возможность их совместного использования в рамках единого процесса анализа. Именно поэтому далее будет рассмотрен набор программных средств, который наилучшим образом соответствует специфике OLAP-данных и задачам ресторанной аналитики.

Разрабатываемая система представляет собой аналитический контур, который объединяет хранение данных, их подготовку, исследование и интеллектуальную обработку в рамках единого процесса. В центре этой концепции находится не отдельный алгоритм и не набор разрозненных таблиц, а последовательная работа с бизнес-показателями ресторанной деятельности. Иными словами, система должна превращать накопленные сведения о работе предприятия в основу для анализа, интерпретации и прогноза.

Логика ее функционирования строится поэтапно. Сначала система получает данные из корпоративного хранилища, где уже собраны сведения о продажах, заказах, позициях меню, филиалах, выручке, затратах, времени обслуживания и других характеристиках. Затем она приводит эти массивы к согласованному виду — устраняет пропуски, проверяет корректность структуры, объединяет связанные

сущности и формирует признаки, пригодные для дальнейшего исследования. На этом этапе сырые записи перестают быть просто архивом операций и превращаются в аналитическую основу, с которой уже можно работать осмысленно.

После подготовки начинается содержательная часть. Система исследует показатели, выявляет закономерности, сопоставляет значения за разные периоды, фиксирует отклонения и позволяет увидеть внутренние связи между экономическими и операционными параметрами. Здесь особенно важно, что анализ не ограничивается сухим перечислением чисел. Напротив, система должна помогать находить ответы на прикладные вопросы: как меняется спрос, какие факторы влияют на выручку, когда возникают пиковые нагрузки, какие категории товаров формируют наибольший вклад в финансовый результат, где проявляются аномалии и скрытые тенденции.

Чтобы такие выводы не оставались абстрактными, концепция системы включает визуальный уровень. Графическое представление делает сложные зависимости заметнее — динамика во времени, распределение значений, различия между объектами и сезонные колебания читаются быстрее, чем длинные табличные выборки. За счет этого аналитическая среда выполняет сразу две функции: она помогает исследовать информацию и одновременно упрощает интерпретацию результатов для пользователя.

Однако ключевая особенность системы состоит в том, что она не останавливается на описательном анализе. Следующий уровень — интеллектуальная обработка данных с применением методов машинного обучения. На основе накопленной истории система может строить модели, которые распознают повторяющиеся паттерны, прогнозируют будущие значения, делят объекты на группы по схожим признакам или оценивают влияние различных факторов на итоговые показатели.

2.1. Обзор и сравнение СУБД как источника данных

Выбор системы управления базами данных в данной работе определяется не формальным предпочтением той или иной платформы, а требованиями самой аналитической задачи. Разрабатываемое решение должно опираться на хранилище, которое не только сохраняет большие массивы структурированной информации, но и позволяет быстро извлекать их для последующей обработки, визуального исследования и построения моделей машинного обучения. По этой причине в качестве возможных вариантов целесообразно рассмотреть несколько распространенных СУБД, которые применяются в корпоративной среде и поддерживают работу с аналитическими нагрузками.

В рамках сравнения рассматриваются Microsoft SQL Server, PostgreSQL и Oracle Database. Каждая из этих платформ решает задачи хранения и выборки данных, однако различается по уровню интеграции с корпоративной инфраструктурой, особенностям администрирования, требованиям к сопровождению и удобству использования в прикладных аналитических проектах. Чтобы сделать выбор обоснованным, необходимо сопоставить их не в общем виде, а применительно к задаче анализа данных ресторанного бизнеса, где важны масштабируемость, устойчивость и предсказуемость работы с крупными табличными массивами.

Прежде всего внимание следует уделить поддержке аналитических запросов. Для рассматриваемой предметной области база данных должна эффективно обрабатывать выборки по временным периодам, филиалам, категориям блюд, продажам, выручке и другим бизнес-показателям. Microsoft SQL Server предоставляет развитые средства выполнения сложных SQL-запросов, хорошо работает с агрегированием, фильтрацией и многомерным анализом. PostgreSQL также демонстрирует высокую гибкость при построении аналитических запросов и активно используется в проектах, где важны расширяемость и открытая архитектура. Oracle Database, в свою очередь, традиционно ориентируется на крупные корпоративные среды и предлагает мощный инструментарий для

обработки масштабных массивов данных, однако ее использование обычно связано с более высокой сложностью внедрения и эксплуатации.

Следующий критерий — интеграция с OLAP-структурами и корпоративной аналитикой. Для данной ВКР этот аспект играет ключевую роль, поскольку анализ ведется на основе данных, уже накопленных в аналитически ориентированном хранилище. Microsoft SQL Server в подобных сценариях выглядит особенно уместно: он широко применяется в корпоративных информационных системах, хорошо вписывается в экосистему бизнес-аналитики и поддерживает стабильную работу с крупными наборами структурированных сведений. PostgreSQL тоже способен выступать источником данных для аналитических решений, однако чаще требует более гибкой, иногда более сложной настройки окружения. Oracle Database исторически сильна в больших корпоративных проектах, но ее инфраструктурная тяжеловесность далеко не всегда оправдана в рамках прикладной разработки, ориентированной на конкретную исследовательскую задачу.

Отдельно следует рассмотреть производительность при работе с большими объемами данных. Для ресторанной аналитики это важно, поскольку даже одна сеть заведений накапливает значительное количество записей — по заказам, продажам, сменам, товарным позициям, финансовым операциям и временным интервалам. MS SQL Server обеспечивает устойчивую обработку подобных массивов, поддерживает индексацию, оптимизацию выполнения запросов и инструменты, которые позволяют контролировать поведение системы под нагрузкой. PostgreSQL также показывает хорошие результаты и заслуженно считается производительной платформой, особенно при грамотной настройке. Oracle Database обладает очень высоким потенциалом в части масштабирования и надежности, но в учебном и прикладном контексте эти преимущества нередко сопровождаются избыточной сложностью и повышенными требованиями к сопровождению.

С точки зрения совместимости с Python все три платформы могут использоваться в аналитических проектах, однако степень удобства их подключения различается. Microsoft SQL Server уверенно интегрируется с Python

через распространенные драйверы и библиотеки доступа к данным, что особенно важно для построения единой цепочки — от SQL-запроса до машинного обучения. PostgreSQL также хорошо взаимодействует с Python и широко применяется в связке с аналитическими инструментами. Oracle Database поддерживает аналогичный сценарий, но настройка соединения и сопровождение такой интеграции часто оказываются более требовательными.

Наконец, следует учитывать сложность развертывания и дальнейшего сопровождения. Для исследовательского проекта предпочтение обычно отдают решению, которое сочетает достаточную функциональность с умеренными затратами на настройку и поддержку. В этом отношении Microsoft SQL Server занимает сбалансированную позицию: он предоставляет развитые возможности корпоративного уровня, при этом сохраняет удобство эксплуатации и предсказуемость работы в прикладных аналитических задачах. PostgreSQL привлекает своей открытостью и гибкостью, но в ряде сценариев требует более тщательной технической настройки. Oracle Database остается мощной промышленной платформой, однако для рассматриваемой задачи ее потенциал может оказаться избыточным по отношению к сложности и ресурсным требованиям.

Таблица 2.1 Сравнение платформ СУБД

Критерий	Microsoft SQL Server	PostgreSQL	Oracle Database
Поддержка аналитических запросов	Высокая — эффективная работа с агрегатами, оконными функциями, сложными выборками	Высокая — гибкий SQL, поддержка расширений для аналитики	Очень высокая — развитые механизмы аналитической обработки
Интеграция с OLAP и корпоративной аналитикой	Тесная интеграция с корпоративными BI-решениями, широко используется в аналитических системах	Возможна, но требует дополнительной настройки и инструментов	Глубокая интеграция в корпоративных системах, особенно в крупных организациях
Производительность при больших объемах данных	Высокая — оптимизация запросов, индексация, стабильная работа под нагрузкой	Высокая — хорошая масштабируемость при правильной настройке	Очень высокая — ориентирована на большие корпоративные нагрузки
Средства администрирования	Удобные встроенные инструменты управления, понятный интерфейс	Гибкое управление, но требует более глубоких технических знаний	Мощные, но сложные инструменты администрирования
Совместимость с Python	Высокая — поддержка через драйверы и библиотеки (ODBC, ORM)	Высокая — активно используется в Python-экосистеме	Есть поддержка, но настройка сложнее

Сложность развертывания и сопровождения	Средняя — баланс между функциональностью и простотой использования	Средняя/выше средней — требует настройки и опыта	Высокая — сложная установка и сопровождение
Распространенность в корпоративной среде	Очень высокая — широко применяется в бизнес-аналитике	Высокая — популярен в open-source проектах	Высокая — используется в крупных корпоративных системах
Гибкость и расширяемость	Ограниченная по сравнению с open-source решениями	Очень высокая — поддержка расширений и кастомизации	Высокая, но в рамках экосистемы Oracle

Проведенное сравнение показывает, что Microsoft SQL Server наиболее полно соответствует условиям проекта. Эта СУБД поддерживает сложные аналитические запросы, хорошо интегрируется с корпоративной OLAP-средой, обеспечивает надежную работу с крупными объемами структурированных данных, предлагает удобные средства администрирования и без существенных затруднений взаимодействует с Python-инструментами. С учетом того, что в данной работе именно MS SQL Server уже используется как базовая система хранения данных, его выбор становится не только логичным, но и методически обоснованным. Для задачи анализа ресторанных показателей такое решение обеспечивает устойчивую основу, на которой можно последовательно выстроить весь дальнейший аналитический контур.

2.2. Обзор и сравнение средств доступа к СУБД из Python

После выбора системы хранения встает следующий вопрос — каким образом прикладная часть будет обращаться к базе данных. Для данной работы этот момент нельзя считать второстепенным. От него зависит не только факт подключения к MS SQL Server, но и то, насколько удобно получится извлекать выборки, передавать их в аналитический контур, сопровождать код и развивать проект без постоянного переписывания уже готовых фрагментов.

В качестве основных средств доступа к СУБД из Python целесообразно рассмотреть SQLAlchemy, pyodbc и pymssql. Эти инструменты решают сходную задачу, однако делают это по-разному. Один дает разработчику более высокий уровень абстракции, другой обеспечивает прямой доступ к серверу, третий привлекает простотой в отдельных сценариях. Поэтому сравнивать их стоит не в отрыве от практики, а через призму конкретной системы, где данные нужно не

просто считать из базы, а встроить в непрерывную цепочку — от SQL-запроса до обработки, визуализации и построения моделей машинного обучения.

`pyodbc` представляет собой низкоуровневый драйвер, который позволяет подключаться к реляционным базам данных через ODBC. Его сильная сторона очевидна: он дает прямой и прозрачный доступ к SQL Server. Разработчик сам формирует строку соединения, отправляет запрос, получает результат и далее управляет всеми этапами работы практически вручную. Такой подход удобен там, где важен полный контроль над взаимодействием с сервером. Однако эта же особенность постепенно превращается в ограничение. Как только проект выходит за рамки нескольких простых скриптов, код начинает обрастать однотипной логикой — курсоры, параметры, соединения, обработка результатов. Структура становится тяжелее, а сопровождение требует большего внимания.

`pymsql` решает ту же задачу более прямолинейно и в ряде случаев действительно позволяет быстро наладить подключение к Microsoft SQL Server. Для несложных операций чтения и выполнения SQL-команд этого может оказаться достаточно. Тем не менее в более современных аналитических проектах данный инструмент используют заметно реже. Причина связана не с базовой функциональностью как таковой, а с тем, что он уступает альтернативам по универсальности, распространенности и общей устойчивости в рамках активно развивающейся Python-экосистемы. Иными словами, для локальных или небольших прикладных задач такой вариант остается рабочим, но в качестве основы для масштабируемой аналитической системы он выглядит менее убедительно.

На этом фоне `SQLAlchemy` занимает иную позицию. Он не ограничивается ролью драйвера и не сводит работу с БД к ручному выполнению команд. Напротив, этот инструмент предлагает более высокий уровень организации доступа к данным. Разработчик может работать и с текстовыми SQL-запросами, и с ORM-подходом, связывая программные сущности со структурой таблиц. Благодаря этому код приобретает четкую архитектуру, а сама логика обращения к базе становится понятнее. Для проекта, в котором важно не эпизодическое чтение данных, а

полноценная интеграция хранилища с аналитическим модулем, такое преимущество оказывается принципиальным.

Если сначала посмотреть на простоту подключения к MS SQL Server, то здесь наиболее прямым решением выглядит `pyodbc`. Он быстро устанавливает соединение и не требует сложного промежуточного слоя. `pymsql` тоже позволяет решить эту задачу без особых затруднений, хотя применяется заметно реже. `SQLAlchemy` требует чуть более внимательной начальной настройки, зато уже на следующем этапе выигрывает за счет более стройной структуры проекта. В результате небольшая дополнительная сложность на старте компенсируется удобством дальнейшей работы.

Разница особенно заметна при сравнении поддержки SQL-запросов и ORM-модели. `pyodbc` и `pymsql` ориентируются прежде всего на прямое выполнение SQL-команд. Такой способ вполне уместен, когда разработчик решает узкую задачу и не планирует усложнять архитектуру. Но если проект развивается, количество запросов растет, появляются новые сущности, связи и уровни логики, ручной подход начинает мешать. `SQLAlchemy` снимает эту проблему. Он сохраняет возможность работать с «чистым» SQL там, где это необходимо, но одновременно открывает доступ к ORM-механизму. За счет этого решение становится гибче и лучше подходит для системной разработки.

Гибкость настройки соединения также имеет значение. Здесь речь идет уже не только о строке подключения, но и о контроле сессий, параметров, устойчивости работы при повторных обращениях к серверу. `pyodbc` дает хороший контроль, однако требует, чтобы разработчик самостоятельно выстраивал значительную часть этой логики. `pymsql` предлагает более простой путь, но уступает в универсальности. `SQLAlchemy` выглядит наиболее сбалансированно — особенно в том случае, когда он работает совместно с `pyodbc`. Такая связка объединяет надежность драйвера и более высокий уровень программной организации.

Особую роль в рамках данного проекта играет совместимость с `pandas`. База данных сама по себе не является конечной точкой аналитического процесса — после загрузки сведения нужно превратить в удобную табличную структуру,

очистить, агрегировать, преобразовать и передать дальше. `pyodbc` позволяет решить эту задачу, но взаимодействие обычно строится в более процедурном стиле. `pymsql` тоже поддерживает подобный сценарий, хотя используется для этого заметно реже. `SQLAlchemy` встраивается в типичную логику аналитической разработки гораздо естественнее: результат запроса быстро переходит в формат, с которым затем удобно работать на последующих этапах.

Если оценивать удобство сопровождения, преимущество `SQLAlchemy` становится еще отчетливее. Для небольших сценариев низкоуровневый драйвер действительно может показаться проще. Однако по мере роста проекта ручная реализация запросов, соединений и обработки результатов начинает усложнять каждое изменение. Код хуже читается, повторяющиеся конструкции множатся, а архитектура постепенно теряет ясность. `SQLAlchemy`, напротив, помогает выстроить доступ к данным более аккуратно — с понятным разделением логики подключения, выполнения запросов и взаимодействия с сущностями предметной области. Для ВКР это важно не только с практической точки зрения, но и с методической: такая структура легче описывается, аргументируется и поддерживается.

Что касается производительности и устойчивости, все три инструмента способны выполнять базовые операции подключения и чтения из `SQL Server`. Но в прикладной аналитике скорость одного запроса — не единственный показатель. Нам важна общая надежность решения, его предсказуемость и пригодность для постоянной работы в составе единой системы. `pyodbc` давно зарекомендовал себя как стабильный драйвер. `pymsql` в этом аспекте выглядит слабее — прежде всего из-за меньшей актуальности и более ограниченного распространения. `SQLAlchemy` работает на более высоком уровне, но в сочетании с `pyodbc` формирует устойчивую и удобную связку, которая особенно хорошо подходит для аналитических проектов.

Таблица 2.2 Сравнение средств доступа к `MS SQL Server` из `Python`

Критерий	<code>SQLAlchemy</code>	<code>pyodbc</code>	<code>pymsql</code>
Простота подключения к <code>MS SQL Server</code>	Высокая, но требует первичной настройки	Высокая — прямое подключение через <code>ODBC</code>	Высокая в простых сценариях
Поддержка SQL-запросов	Полная	Полная	Полная

Поддержка ORM-подхода	Да	Нет	Нет
Гибкость настройки соединения	Высокая	Высокая	Средняя
Совместимость с pandas	Высокая	Высокая	Достаточная
Удобство сопровождения кода	Высокое	Среднее	Среднее
Производительность	Высокая, особенно в связке с pyodbc	Высокая	Достаточная
Стабильность и распространенность	Высокая	Высокая	Ниже, чем у альтернатив
Пригодность для масштабируемого проекта	Очень высокая	Средняя	Ограниченная

Для разрабатываемой системы целесообразно выбрать SQLAlchemy как основной инструмент доступа к базе данных. Он делает архитектуру проекта более гибкой, упорядоченной и удобной для развития. pyodbc логично использовать как надежный драйвер, который обеспечивает техническую сторону подключения к MS SQL Server. pymssql сохраняет значение как возможная альтернатива, однако в условиях данного проекта он не дает тех преимуществ, которые могли бы перевесить выбор в пользу более универсального и устойчивого решения.

2.3. Моделирование базы данных

Для анализа данных в ресторанном бизнесе обычной транзакционной базы недостаточно. Она хорошо поддерживает повседневные операции — оформление заказа, оплату, списание ингредиентов, — однако плохо подходит для поиска закономерностей, сравнения периодов и построения прогнозов. Поэтому в работе используется хранилище данных, организованное по OLAP-подходу.

Если транзакционная система фиксирует событие, то аналитическая — помогает его интерпретировать. В этом и заключается ключевое различие. OLTP-среда отвечает за скорость операций, тогда как OLAP-структура ориентируется на сложные выборки, агрегирование и сравнение показателей в разрезе разных факторов. Для ресторанной сферы это особенно важно, потому что спрос здесь зависит сразу от нескольких переменных: времени суток, дня недели, сезона, состава меню, типа клиента, филиала и способа оформления заказа.

В качестве основы выбрана схема «звезда». Она наглядно отражает логику предметной области: в центре находится таблица фактов, а вокруг нее располагаются измерения, которые задают контекст для анализа. Такой подход упрощает построение отчетов, делает структуру понятной и ускоряет обработку аналитических запросов.

Центральным процессом в ресторанном бизнесе выступает продажа. Именно поэтому ядром модели становится факт реализации позиции заказа. Иначе говоря, каждая запись в таблице фактов соответствует одной строке чека — конкретному блюду или напитку, проданному в составе определенного заказа. Этот уровень детализации выбран осознанно. Он позволяет анализировать не только заказ целиком, но и поведение отдельных товарных позиций. За счет этого можно определить, какие блюда формируют основную выручку, какие продаются нестабильно, а какие лучше реагируют на скидки или сезонные изменения.

Подобная детализация важна и для последующей интеллектуальной обработки. Модель машинного обучения не работает с абстрактной «продажей вообще» — ей нужны наблюдения, из которых можно выделить устойчивые признаки. Если хранить сведения только в агрегированном виде, значительная часть полезной информации теряется. Напротив, детализированный уровень дает возможность позже собрать любые нужные укрупнения — по дням, филиалам, категориям меню, периодам активности клиентов или каналам оформления заказа.

Основу хранилища составляет таблица фактов `Fact_Sales`. В ней концентрируются количественные показатели, связанные с продажей: число единиц товара, цена, скидка, себестоимость, итоговая выручка, прибыль, длительность приготовления и обслуживания. Помимо самих метрик, в ней хранятся ссылки на измерения, которые позволяют рассматривать продажу с разных сторон. Таким образом, запись в таблице фактов отвечает сразу на два вопроса: что произошло и в каком контексте это произошло.

Таблица 2.3 `Fact_Sales`

Атрибут	Тип данных	Назначение
<code>Sales_Fact_ID</code>	BIGINT	Уникальный идентификатор записи

Date_ID	INT	Ссылка на дату продажи
Time_ID	INT	Ссылка на время суток
Restaurant_ID	INT	Идентификатор филиала
Menu_Item_ID	INT	Идентификатор блюда
Customer_ID	INT	Идентификатор клиента
Quantity	INT	Количество единиц
Unit_Price	DECIMAL(10,2)	Цена за единицу
Discount_Amount	DECIMAL(10,2)	Сумма скидки
Cost_Amount	DECIMAL(10,2)	Себестоимость
Revenue_Amount	DECIMAL(10,2)	Выручка
Profit_Amount	DECIMAL(10,2)	Прибыль
Preparation_Time	INT	Время приготовления (сек/мин)
Service_Time	INT	Длительность обслуживания
Rating_Value	DECIMAL(3,2)	Оценка клиента

Чтобы анализ не ограничивался простым подсчетом выручки, вокруг таблицы фактов формируется набор измерений. Первое и самое важное из них — время. Без временного разреза невозможно исследовать ни динамику продаж, ни сезонные колебания, ни изменение клиентской активности. В измерении времени фиксируются дата, день недели, месяц, квартал, год, признак выходного или праздничного дня, а при необходимости — и сезон. Такая структура помогает проследивать как краткосрочные изменения, так и долгие тенденции.

Таблица 2.4 Dim_Date

Атрибут	Тип данных	Назначение
Date_ID	INT	Ключ записи
Full_Date	DATE	Полная дата
Day_Number	INT	День месяца
Month_Number	INT	Номер месяца
Month_Name	VARCHAR(20)	Название месяца
Quarter_Number	INT	Квартал
Year_Number	INT	Год
Day_Of_Week	INT	День недели
Is_Weekend	BOOLEAN	Признак выходного
Is_Holiday	BOOLEAN	Праздничный день
Season	VARCHAR(20)	Сезон

Таблица 2.5 Dim_Time

Атрибут	Тип данных	Назначение
Time_ID	INT	Ключ

Hour	INT	Час
Minute	INT	Минута
Time_Interval	VARCHAR(20)	Интервал (например, «утро», «вечер»)
Part_Of_Day	VARCHAR(20)	Часть суток

Следующее важное измерение связано с филиалом или точкой продаж. Даже если сеть работает в одном городе, разные рестораны показывают неодинаковые результаты. На выручку влияют район, формат заведения, количество посадочных мест, наличие доставки и другие характеристики. Измерение ресторана дает возможность сравнивать подразделения между собой, искать сильные и слабые точки, а также выявлять территориальные различия в спросе.

Таблица 2.6 Dim_Restaurant

Атрибут	Тип данных	Назначение
Restaurant_ID	INT	Ключ
Restaurant_Name	VARCHAR(100)	Название
City	VARCHAR(50)	Город
District	VARCHAR(50)	Район
Address	VARCHAR(200)	Адрес
Restaurant_Type	VARCHAR(50)	Тип заведения
Opening_Date	DATE	Дата открытия
Seating_Capacity	INT	Вместимость
Delivery_Available	BOOLEAN	Доступность доставки
Drive_Thru_Available	BOOLEAN	Наличие drive-thru

Не менее значимым остается измерение блюда. Именно оно раскрывает ассортиментную структуру бизнеса. Для каждой позиции можно сохранить название, категорию, подкатегорию, тип кухни, стандартную цену, себестоимость, признак сезонности и дату появления в меню. Благодаря этому аналитика выходит за рамки общего оборота и показывает внутреннюю механику продаж. Становится видно, какие блюда стабильно удерживают интерес гостей, какие работают как имиджевые позиции, а какие только создают нагрузку без заметного коммерческого эффекта.

Таблица 2.7 Dim_Menu_Item

Атрибут	Тип данных	Назначение
Menu_Item_ID	INT	Ключ
Menu_Item_Name	VARCHAR(100)	Название блюда
Category_Name	VARCHAR(50)	Категория
Subcategory_Name	VARCHAR(50)	Подкатегория
Cuisine_Type	VARCHAR(50)	Тип кухни
Portion_Size	VARCHAR(50)	Размер порции
Calories	INT	Калорийность
Base_Cost	DECIMAL(10,2)	Себестоимость
Standard_Price	DECIMAL(10,2)	Базовая цена
Is_Seasonal	BOOLEAN	Сезонность
Launch_Date	DATE	Дата появления
Is_Active	BOOLEAN	Актуальность

Еще один важный блок — сведения о клиентах. В аналитической модели они особенно ценны, поскольку позволяют связать продажи с потребительским

поведением. Здесь могут использоваться обезличенные идентификаторы, возрастные группы, статус в программе лояльности, клиентский сегмент, предпочитаемый канал заказа и характеристики среднего чека. Благодаря такому измерению можно проследить, как ведут себя новые и постоянные гости, кто чаще оформляет доставку, а кто предпочитает посещение зала, какие сегменты активнее реагируют на акции, а какие обеспечивают наиболее устойчивую выручку.

Таблица 2.8 Dim_Customer

Атрибут	Тип данных	Назначение
Customer_ID	INT	Ключ
Gender	VARCHAR(10)	Пол
Age_Group	VARCHAR(20)	Возрастная группа
Loyalty_Status	VARCHAR(50)	Статус лояльности
Registration_Date	DATE	Дата регистрации
Preferred_Channel	VARCHAR(50)	Предпочитаемый канал
Segment	VARCHAR(50)	Сегмент
City	VARCHAR(50)	Город
Average_Check_Group	VARCHAR(50)	Группа по среднему чеку

На этой базе становятся доступными типовые OLAP-операции. Можно укрупнять данные и переходить от дней к месяцам, а затем к кварталам. Можно, наоборот, детализировать картину — например, от уровня филиала спускаться к категориям и отдельным блюдам. Можно выделить конкретный срез, взяв только доставку или только выходные дни. Можно комбинировать сразу несколько условий и получать более сложные аналитические выборки. Наконец, можно менять оси представления и смотреть на одни и те же показатели под другим углом. Все это особенно полезно для ресторанного бизнеса, где поведение спроса редко бывает линейным.

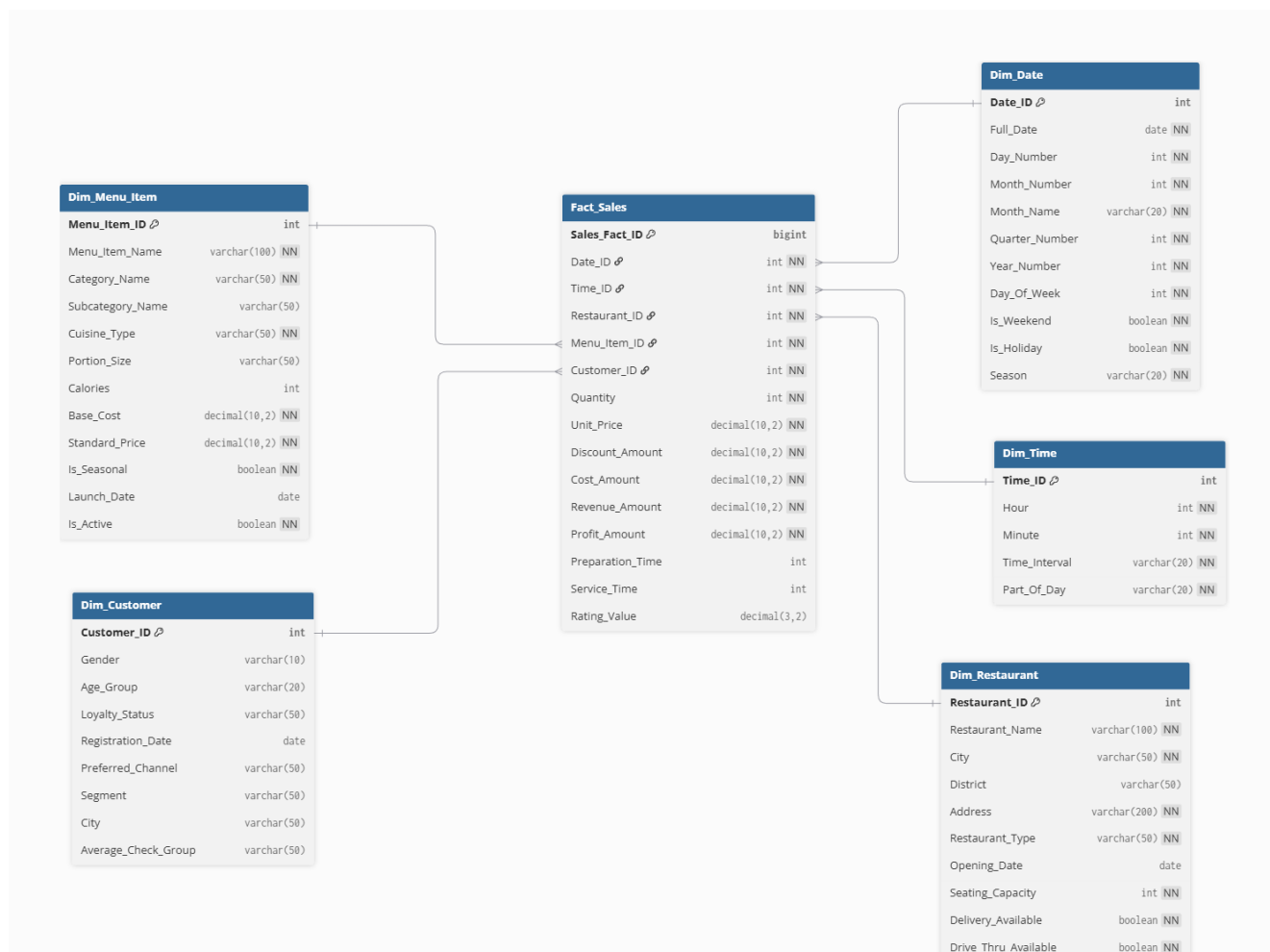


Рисунок 2.1 Архитектура базы данных

На основе таблицы фактов и измерений можно рассчитывать широкий набор метрик: выручку, прибыль, количество проданных позиций, число заказов, средний чек, среднюю скорость обслуживания, долю акционных продаж, вклад каналов сбыта, повторяемость покупок. При необходимости из детализированной структуры формируются агрегированные витрины — например, продажи по дням и филиалам, загрузка по часам, динамика категорий меню или поведение отдельных групп клиентов. Тем не менее именно первичный аналитический слой остается главным источником для всех последующих расчетов.

3. Реализация прототипа аналитической системы

3.1. Реализация хранилища данных

Для создания хранилища данных был разработан Python-скрипт `'create_restaurant_dw_duckdb.py'`. Он выполняет полный цикл подготовки локальной аналитической базы: создает файл DuckDB, очищает ранее созданные объекты, формирует структуру таблиц, добавляет индексы, создает аналитические представления и, при запуске с соответствующим параметром, наполняет базу демонстрационными данными. Скрипт не требует ручного выполнения SQL-команд — вся логика развертывания перенесена в программный код.

В начале файла подключаются библиотеки, которые нужны для работы с базой, датами, денежными значениями и параметрами командной строки. Модуль `'duckdb'` обеспечивает подключение к файловой СУБД, `'argparse'` отвечает за запуск с внешними параметрами, `'datetime'` используется при построении календаря и генерации продаж, а `'Decimal'` применён для расчетов стоимости, выручки, скидок и прибыли. За счет этого финансовые показатели не теряют точность при округлении, что особенно важно для дальнейшей аналитики. По умолчанию скрипт создает базу с именем `'restaurant_dw.duckdb'`. Это значение хранится в константе `'DB_PATH'`, но пользователь может передать другой путь через аргумент `'--db'`. Такой подход делает скрипт удобным для повторного использования: одну и ту же программу можно применять для учебной демонстрации, тестирования дашборда или формирования новой версии базы без изменения исходного кода.

Структура хранилища задается через список `'CREATE_TABLES'`. В него включены SQL-инструкции для создания таблиц измерений и таблицы фактов. Скрипт размещает DDL-команды внутри Python-кода, поэтому схема базы остается частью проекта и воспроизводится одинаково при каждом запуске. В таблицах указаны первичные ключи, внешние ключи и проверочные ограничения. Эти правила защищают базу от некорректных записей: например, количество

проданных блюд не может быть меньше единицы, финансовые значения не принимают отрицательные числа, а рейтинг ограничивается диапазоном от 0 до 5.

Перед созданием новой схемы программа удаляет старые объекты. Для этого используется список `'DROP_ORDER'`, где таблицы расположены в порядке, учитывающем связи между ними. Сначала удаляется таблица фактов, затем справочные таблицы. Такой порядок предотвращает ошибки, которые могли бы возникнуть из-за внешних ключей. Представления удаляются отдельно — до удаления таблиц, поскольку они зависят от уже существующих объектов базы.

Функция `'recreate_schema()'` управляет пересозданием структуры. Она последовательно удаляет представления, очищает набор таблиц, выполняет SQL-команды из `'CREATE_TABLES'`, затем создает индексы и витрины. В результате скрипт каждый раз приводит базу к предсказуемому состоянию. Это важно для разработки: при изменении структуры можно заново запустить программу и получить актуальную версию хранилища без ручной очистки файла.

После создания таблиц скрипт добавляет индексы. Они описаны в списке `'CREATE_INDEXES'` и ориентированы прежде всего на таблицу `'Fact_Sales'`. Индексы создаются по ключам даты, времени, ресторана, блюда и клиента. Отдельно добавлен составной индекс по полям `'Date_ID'`, `'Restaurant_ID'` и `'Menu_Item_ID'`, поскольку аналитические запросы часто обращаются к продажам в разрезе периода, филиала и позиции меню. Это решение ускоряет соединение таблицы фактов с измерениями и снижает нагрузку при выполнении выборок. Кроме таблиц и индексов, скрипт формирует несколько представлений. Они находятся в списке `'CREATE_VIEWS'` и служат готовыми аналитическими срезами. Представление `'vw_daily_sales'` агрегирует продажи по датам и ресторанам, `'vw_menu_sales'` группирует показатели по блюдам и категориям, `'vw_restaurant_menu'` показывает состав меню по филиалам, а `'vw_customer_flow'` собирает сведения о потоке клиентов, количестве проданных позиций, выручке и прибыли. Благодаря этим объектам последующие модули проекта могут обращаться не только к исходным таблицам, но и к уже подготовленным агрегированным данным.

Отдельный блок скрипта отвечает за календарные и временные признаки. Функция ``get_season()`` определяет сезон по номеру месяца, ``get_part_of_day()`` относит час к утру, дню, вечеру или ночи, а ``get_time_interval()`` формирует получасовой интервал. Вспомогательные функции ``date_id()`` и ``time_id()`` превращают дату и время в числовые идентификаторы. Например, дата кодируется в формате ``YYYYMMDD``, а время — через сочетание часа и минуты. Благодаря этому календарные и временные записи получают устойчивые ключи, понятные и человеку, и программе. Функция ``fill_date_dimension()`` заполняет таблицу дат. Она принимает начальную и конечную даты, проходит по каждому дню выбранного периода и формирует строку с календарными атрибутами: номером дня, месяцем, названием месяца, кварталом, годом, днем недели, признаком выходного и сезоном. По умолчанию скрипт строит календарь с 1 января 2024 года по 31 декабря 2026 года, однако эти границы можно изменить через параметры ``--start-date`` и ``--end-date``.

Таблица времени наполняется функцией ``fill_time_dimension()``. Она перебирает все часы и минуты суток, создает 1440 записей и добавляет для каждой из них временной интервал и часть дня. В результате база получает полную минутную сетку, которую можно использовать для анализа внутридневной нагрузки. Эта детализация помогает исследовать пики посещаемости, распределение заказов по часам и различия между обеденным и вечерним спросом.

Для демонстрационного режима в скрипте предусмотрена функция ``fill_demo_data()``. Она создает набор ресторанов, меню, клиентов и продаж. Генератор случайных чисел инициализируется фиксированным значением ``42``, поэтому при повторном запуске программа формирует воспроизводимый набор данных. Это удобно при тестировании: дашборд и модели машинного обучения получают одинаковую основу, а результаты можно сравнивать между запусками без влияния случайных изменений. Сначала демонстрационный блок добавляет три ресторана с разными характеристиками: районом, форматом, вместимостью, наличием доставки и drive-thru. Затем скрипт формирует меню для каждого филиала. В карточке блюда учитываются категория, подкатегория, кухня, размер

порции, калорийность, себестоимость, стандартная цена, сезонность и дата запуска. Для сезонных позиций указывается период, в который спрос на блюдо должен возрасти.

После меню создается клиентская база. Скрипт генерирует 500 покупателей и распределяет между ними пол, возрастную группу, статус лояльности, предпочтительный канал, сегмент и группу среднего чека. Вероятности заданы не равномерно: например, постоянные клиенты и участники программ лояльности встречаются чаще, чем редкие сегменты. За счет этого демонстрационные данные выглядят ближе к реальной бизнес-среде, где покупатели отличаются частотой посещений и реакцией на скидки. При генерации заказа скрипт выбирает клиента с учетом статуса лояльности: покупатели с более высоким уровнем получают больший вес, поэтому чаще попадают в продажи. Время визита также рассчитывается неравномерно. Основная доля заказов приходится на обеденный и вечерний периоды, что соответствует логике ресторанного бизнеса. Затем программа определяет количество строк в заказе, выбирает блюда, рассчитывает количество порций, скидку, выручку, себестоимость, прибыль, время приготовления, время обслуживания и рейтинг.

Главная функция сборки базы называется `'build_database()'`. Она получает путь к файлу, границы календаря и признак демонстрационного заполнения. Внутри функция открывает соединение с DuckDB, вызывает пересоздание схемы, заполняет календарь и время, а при наличии флага `'--demo'` добавляет рестораны, меню, клиентов и продажи. После завершения загрузки выполняется команда `'CHECKPOINT'`, которая фиксирует изменения в файле базы.

Запуск скрипта организован через функцию `'main()'`. Она описывает параметры командной строки: `'--db'` для выбора файла базы, `'--start-date'` и `'--end-date'` для настройки календаря, `'--demo'` для включения генерации тестовых данных. После чтения аргументов программа передает их в `'build_database()'`. Например, для создания демонстрационной базы достаточно выполнить команду:

```
python create_restaurant_dw_duckdb.py --demo
```

При необходимости можно указать собственное имя файла и период календаря: `python create_restaurant_dw_duckdb.py --db restaurant_dw.duckdb --start-date 2024-01-01 --end-date 2026-12-31 --demo`

3.2. Примеры аналитических отчетов

После формирования хранилища следующим этапом стала разработка интерактивного веб-интерфейса для анализа ресторанных показателей. За этот этап отвечает модуль `restaurant_ml_dashboard.py`. Он объединяет подключение к базе DuckDB, загрузку сведений из таблицы фактов и измерений, подготовку аналитических выборок, построение графиков и запуск моделей машинного обучения. В коде используются `pandas` для табличной обработки, `Plotly` для визуализации, `Dash` для интерфейса, `SQLAlchemy` для обращения к базе, а также инструменты `scikit-learn` для обучения и сравнения моделей прогноза.

Работа панели начинается с класса `DashboardData`. Он получает путь к файлу хранилища, создает подключение через `create_engine`, затем загружает три набора сведений: детализированные продажи, календарное измерение и справочник ресторанов. Основную выборку формирует SQL-запрос, который соединяет `Fact_Sales` с таблицами `Dim_Date`, `Dim_Restaurant`, `Dim_Menu_Item` и `Dim_Customer`. Благодаря этому веб-интерфейс получает не разрозненные идентификаторы, а уже расшифрованные бизнес-признаки: дату, сезон, день недели, ресторан, район, тип заведения, блюдо, категорию меню, клиентский сегмент, статус лояльности, выручку, прибыль, скидку, рейтинг и прочие поля.

Интерфейс приложения создает функция `create_app()`. Она инициализирует объект `Dash`, задает заголовок страницы и формирует два рабочих раздела: «Многомерный анализ» и «Прогноз продаж». Первый раздел предназначен для исследования фактических показателей, второй — для оценки будущей динамики. Рисунок 3.1 демонстрирует вкладку многомерного анализа: пользователь видит панель фильтров, выбирает период, рестораны, сезоны, метрику, измерения и тип визуализации, после чего график перестраивается без перезапуска приложения.

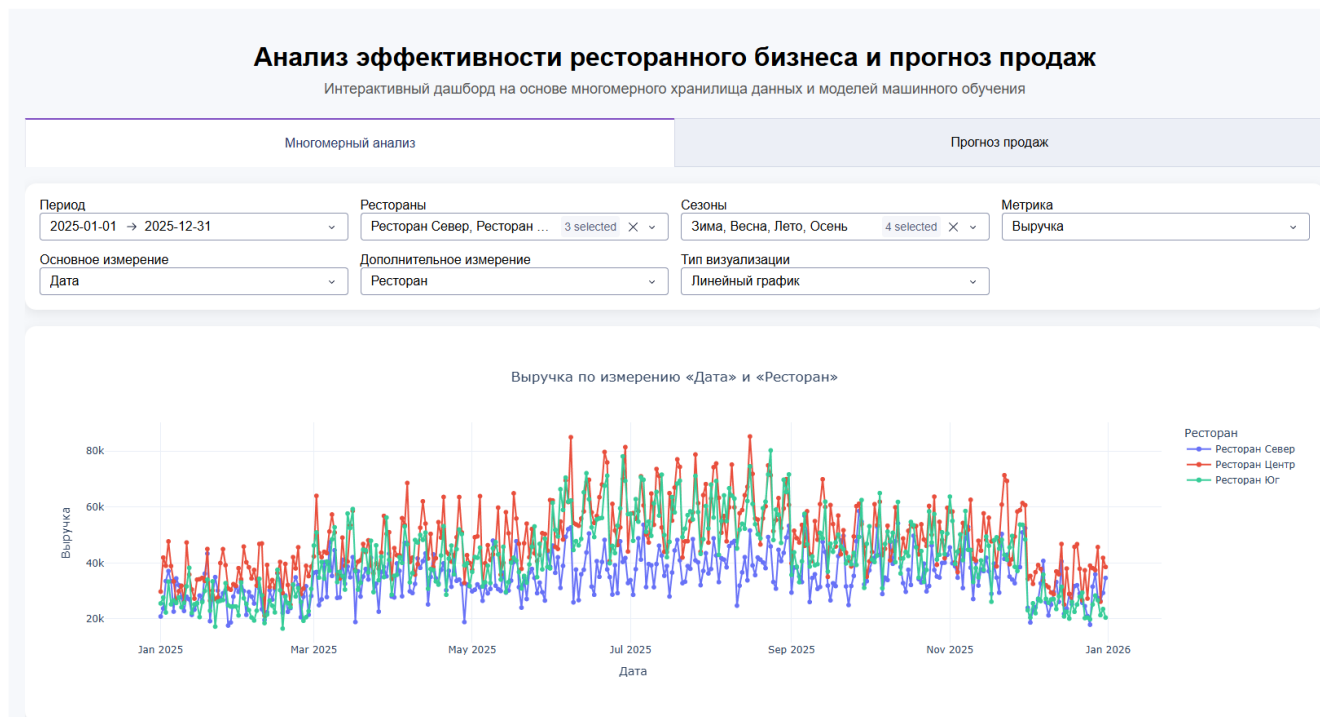


Рисунок 3.1 Интерфейс интерактивной системы

Вкладка многомерного анализа решает задачу гибкого OLAP-исследования. Пользователь может ограничить период через `DatePickerRange`, выбрать один или несколько ресторанов, оставить нужные сезоны, задать основной показатель и изменить аналитический разрез. В качестве метрик код поддерживает выручку, прибыль, количество блюд, скидки, число уникальных клиентов, строки продаж, средний рейтинг и средний чек. Набор измерений шире: дата, ресторан, сезон, месяц, день недели, тип заведения, район, категория блюда, подкатегория, кухня, позиция меню, сезонность блюда, пол клиента, возрастная группа, статус лояльности, канал, сегмент и группа среднего чека. Визуальная структура интерфейса подчинена прикладному сценарию. Сначала пользователь видит фильтры, затем — график, ниже — таблицу с числовой детализацией. На прогнозной вкладке код добавляет индикатор загрузки `dcc.Loading`, поскольку обучение моделей и расчет будущего ряда требуют больше времени, чем обычная агрегация. Карточное оформление, сетка из четырех колонок и единый стиль таблиц делают панель компактной: аналитик не переходит между разными окнами и управляет исследованием из одной рабочей области.

В качестве примера аналитической работы с дашбордом были построены визуализации по клиентскому потоку и сезонной выручке. Первый график

отражает ежедневное количество уникальных клиентов по трем ресторанам. На нем хорошо заметна неравномерность спроса в течение года: в январе и феврале поток остается сравнительно низким, весной начинает расти, а летом достигает максимальных значений. Наиболее высокая посещаемость наблюдается у ресторанов «Центр» и «Юг», тогда как «Север» почти на всем интервале показывает более сдержанную динамику.

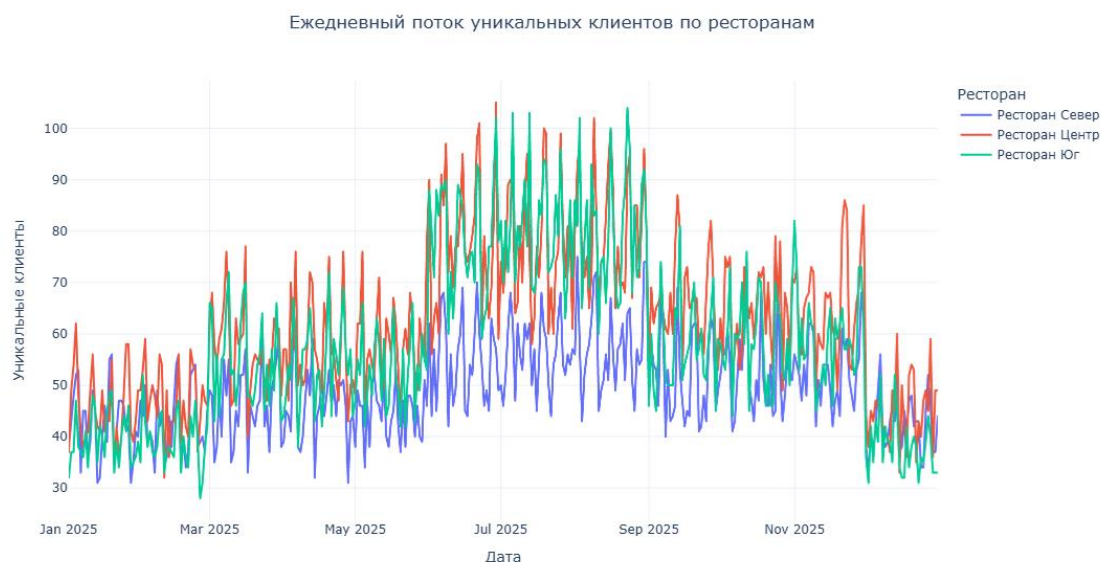


Рисунок 3.2 Ежедневный поток уникальных клиентов по ресторанам

Для снижения влияния случайных дневных всплесков дополнительно использовано 7-дневное скользящее среднее. Эта визуализация делает тренд более читаемым: резкий рост клиентского потока начинается в марте, максимальная нагрузка приходится на летний период, после чего в сентябре происходит заметное снижение. К концу года посещаемость падает почти до уровня начала периода.

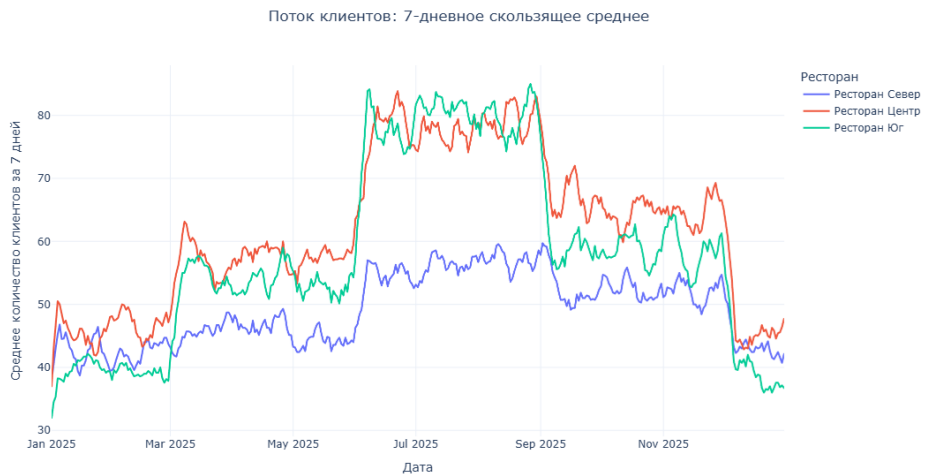


Рисунок 3.3 Поток клиентов: 7-дневное скользящее среднее

Третья визуализация показывает выручку по сезонам и ресторанам. Диаграмма подтверждает, что лето дает наибольший финансовый результат: ресторан «Центр» получает около 5,5 млн, ресторан «Юг» — около 5,4 млн, ресторан «Север» — около 3,6 млн. Зимой показатели ниже: особенно заметно снижение у ресторана «Юг», где выручка составляет около 2,3 млн. В разрезе филиалов лидирует «Центр» во всех сезонах, а «Север» стабильно занимает более слабую позицию по объему продаж.



Рисунок 3.4 Выручка по сезонам и ресторанам

Следующий блок визуализаций раскрывает структуру продаж на уровне меню, категорий и ключевых показателей ресторанной сети. Горизонтальная диаграмма «Топ-15 блюд по выручке» показывает, какие позиции формируют основной денежный поток. Наибольший результат дает «Стейк из лосося» — около 2,5 млн. Среди блюд ресторана «Центр» лидируют «Пицца Маргарита», «Лазанья», «Ризотто с грибами», «Паста Болоньезе» и «Паста Карбонара». У ресторана «Юг» высокие позиции занимают блюда грузинской кухни: шашлык, хачапури, люля-кебаб, чахохбили и хинкали.

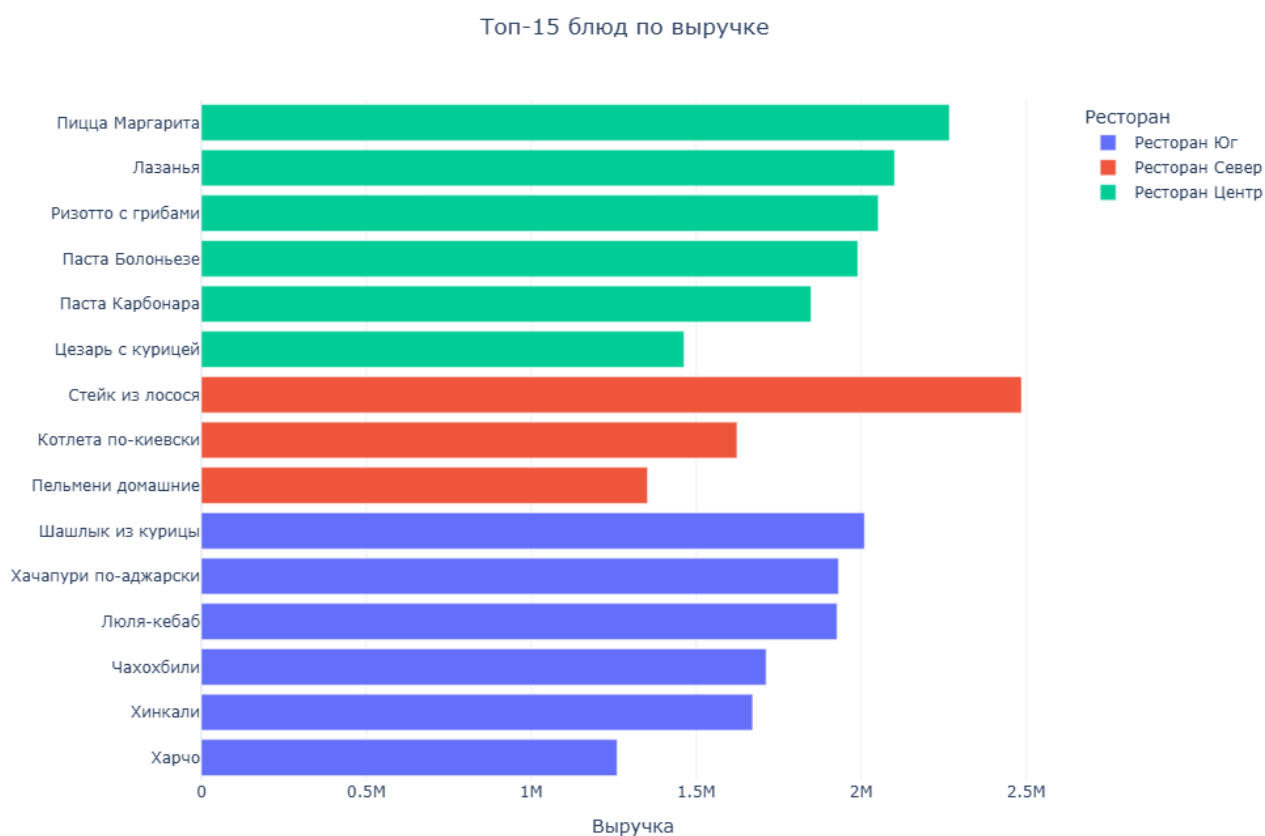


Рисунок 3.5 Топ-15 блюд по выручке

Диаграмма treemap показывает вклад ресторанов и категорий блюд в общую структуру выручки. Наибольшие блоки занимают основные блюда — именно они формируют главную часть продаж во всех филиалах. Ресторан «Центр» и ресторан «Юг» имеют более крупные зоны по сравнению с рестораном «Север», что подтверждает их более сильный вклад в финансовый результат сети. Цветовая шкала дополнительно отражает прибыль: категории с более высокой прибылью

выделяются ярче, поэтому диаграмма одновременно показывает объем продаж и экономическую отдачу.

Структура выручки по ресторанам и категориям блюд



Рисунок 3.6 Структура выручки по ресторанам и категориям блюд

Сравнение сезонных и обычных блюд показывает, что основной оборот создают постоянные позиции меню. Их выручка значительно выше во всех сезонах: зимой — около 7,3 млн, весной — около 11 млн, летом — около 12 млн, осенью — около 11 млн. Сезонные блюда заметнее проявляют себя летом, где их выручка достигает примерно 2,5 млн. Весной показатель сезонных позиций минимален — около 210 тыс.

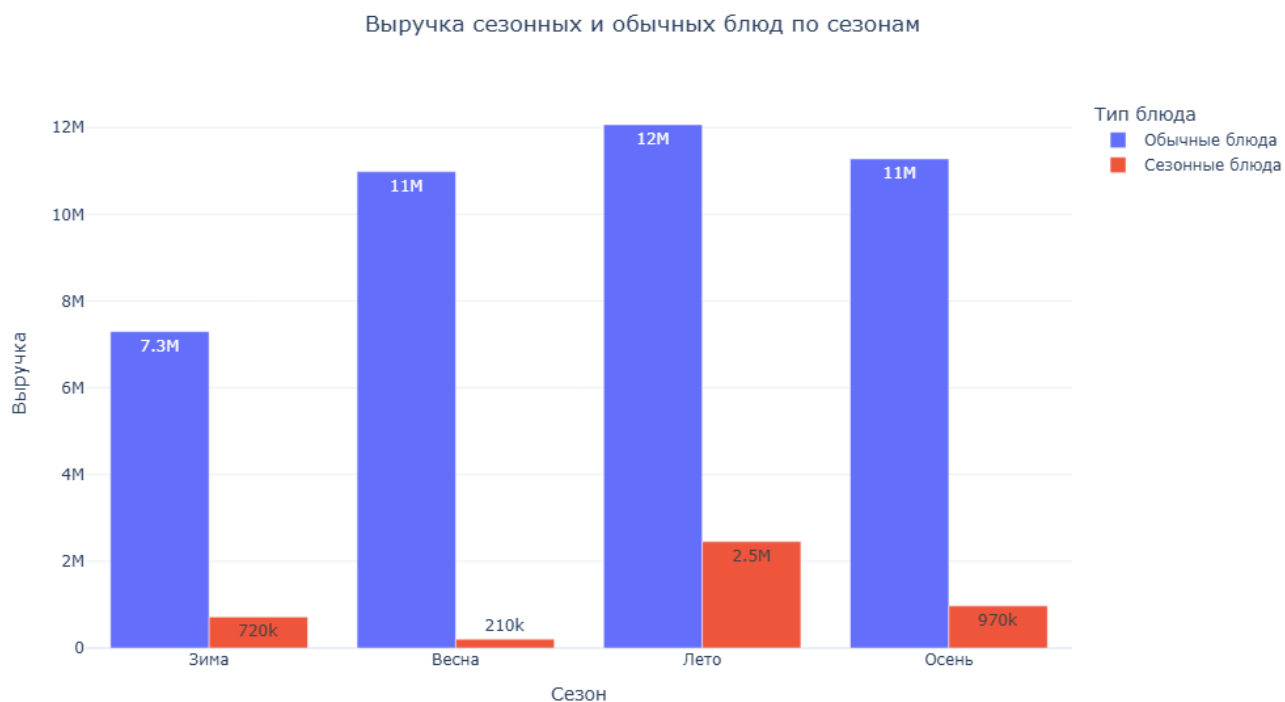


Рисунок 3.7 Выручка сезонных и обычных блюд по сезонам

Диаграмма рентабельности сопоставляет выручку, маржинальность и объем продаж по отдельным блюдам. По оси X расположена выручка, по оси Y — маржинальность, а размер точки отражает количество проданных позиций. В верхней части графика находятся блюда с высокой маржинальностью, например напитки и отдельные позиции с низкой себестоимостью. Справа расположены блюда с большим оборотом, но более умеренной маржинальностью. Аналитик может использовать этот график для деления меню на группы: позиции с высокой прибылью, блюда с высоким оборотом, товары с низкой отдачей и кандидаты на пересмотр цены или рецептуры.

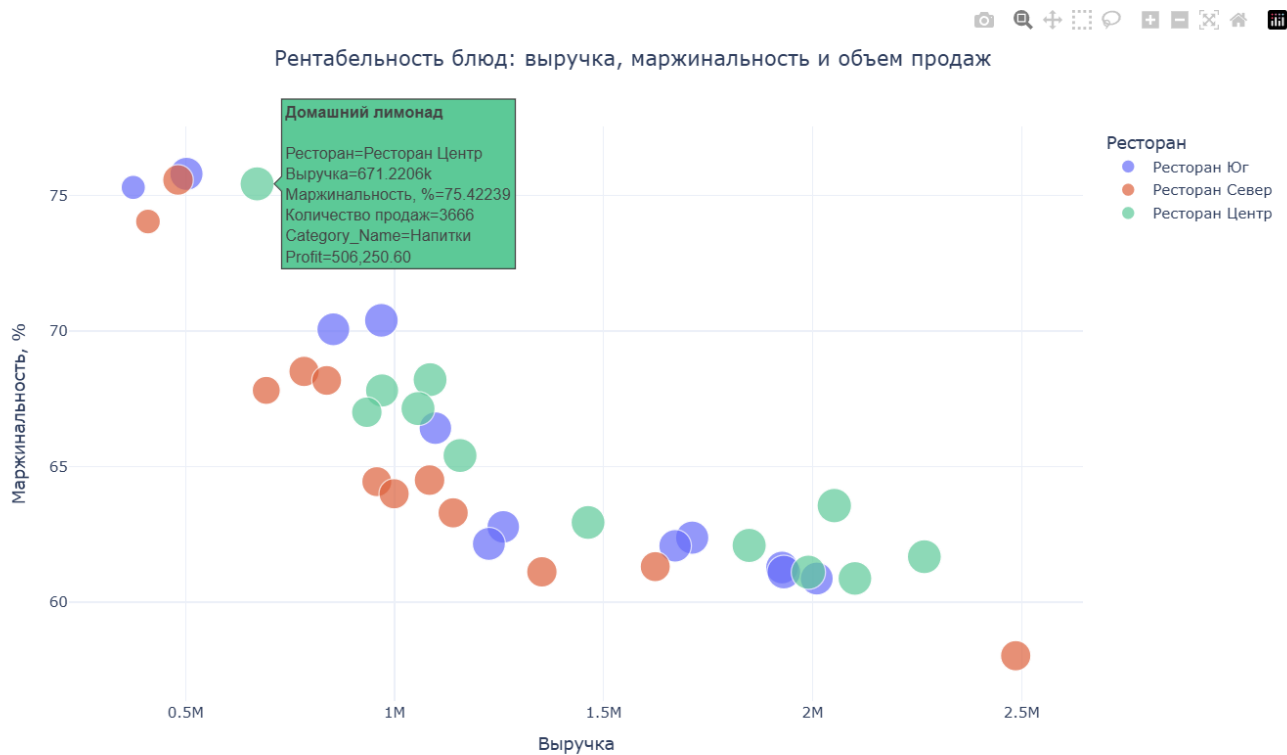


Рисунок 3.8 Рентабельность блюд: выручка, маржинальность и объем продаж



Рисунок 3.9 Основные KPI ресторанной сети

Отдельная KPI-панель показывает укрупненное состояние ресторанной сети. За анализируемый период база содержит 500 уникальных клиентов, объем продаж достигает 115,9 тыс. блюд, совокупная выручка составляет 45 973 847, прибыль — 29 361 577. Средняя маржинальность равна 63,87 %, а средний рейтинг — 4,44. Эти

показатели дают быстрый обзор бизнеса без перехода к детализации: пользователь сразу видит масштаб клиентской базы, общий оборот, прибыльность и уровень клиентской оценки.

3.3. Использование машинного обучения в анализе данных

Перед обучением моделей исходные продажи привели к формату, пригодному для прогнозирования. В качестве первичного источника использовалась таблица фактов Fact_Sales, объединенная с измерениями даты, ресторана, меню и клиента. После выполнения SQL-запроса система получила не набор технических идентификаторов, а расширенную аналитическую таблицу: дата продажи, сезон, день недели, ресторан, район, тип заведения, параметры меню, клиентский сегмент, количество блюд, скидка, выручка, прибыль и рейтинг. Этот набор стал исходной базой для дальнейшего преобразования. Для машинного обучения не использовались отдельные строки чеков, поскольку прогноз должен показывать будущую динамику бизнеса на уровне филиала. Поэтому данные агрегировали до уровня «день — ресторан». На этом уровне рассчитали основные целевые и вспомогательные показатели: количество строк продаж, число уникальных клиентов, объем проданных блюд, выручку, прибыль, сумму скидок и средний рейтинг.

После агрегации была сформирована полная календарная сетка. Скрипт взял диапазон дат, присутствующий в продажах, и соединил его со справочником ресторанов. За счет этого в выборку попали все сочетания даты и филиала, включая дни без продаж. Для количественных полей пропуски заменили нулями, а отсутствующий средний рейтинг — медианным значением. Отдельное значение получили лаговые признаки. Для выбранной целевой переменной система рассчитала значения за 1, 7, 14 и 28 дней до текущей даты. Кроме того, были добавлены скользящие средние и стандартные отклонения за окна 7, 14 и 28 дней.

Затем признаки разделили на две группы. К категориальным отнесли название ресторана, город, район, тип заведения и сезон. В числовую группу вошли календарные параметры, вместимость, признаки доставки и drive-thru, а также все

лаговые и скользящие показатели. Перед передачей в модели категориальные поля кодируются через OneHotEncoder, а пропуски обрабатываются SimpleImputer: для числовых признаков используется медиана, для категориальных — наиболее частое значение.

После подготовки признаков мы обучили несколько моделей для прогноза ключевых ресторанных показателей. В данном примере целевой переменной выступила выручка. Код обучает пять вариантов: Baseline_Median, Ridge, RandomForest, ExtraTrees и HistGradientBoosting. Каждая модель входит в единый Pipeline: сначала препроцессор заполняет пропуски и кодирует категориальные признаки, затем алгоритм выполняет обучение на исторических данных. Последние 60 дней выборки используются как тестовый период, поэтому проверка имитирует реальную задачу — прогноз будущих значений по прошлой динамике. Качество оценивалось по четырем метрикам: MAE, RMSE, MAPE и R^2 . MAE показывает среднюю абсолютную ошибку в денежных единицах, RMSE сильнее реагирует на крупные промахи, MAPE выражает ошибку в процентах, а R^2 отражает долю объясненной изменчивости. В коде прогнозные значения дополнительно ограничиваются снизу нулем, чтобы модель не выдавала отрицательную выручку.

	Model	MAE	RMSE	MAPE	R2
0	ExtraTrees	6093.753911	7473.345082	18.880476	0.558154
1	RandomForest	6866.233232	8298.248861	21.936620	0.455230
2	Ridge	7049.587272	8584.831164	22.828455	0.416952
3	HistGradientBoosting	7219.033201	8710.969916	23.170143	0.399693
4	Baseline_Median	10721.994444	12699.521369	36.025999	-0.275896

Рисунок 3.10 Сравнение качества моделей прогнозирования выручки

Наилучший результат показала модель ExtraTrees. Ее MAE составил около 6 093,75, RMSE — 7 473,35, MAPE — 18,88 %, R^2 — 0,558. Это означает, что модель уловила заметную часть закономерностей в выручке и дала наиболее точный прогноз среди рассмотренных вариантов. По сравнению с медианным ориентиром

RMSE снизился примерно на 41 %, а процентная ошибка уменьшилась с 36,03 % до 18,88 %. Разница подтверждает, что календарные, ресторанные и лаговые признаки действительно несут полезную информацию для прогноза.

На графике «Факт и прогноз на тестовом периоде» видно, что ExtraTrees в целом повторяет направление изменения выручки по ресторанам. Красная пунктирная линия сглаживает резкие дневные скачки, но сохраняет общую траекторию: падение в начале декабря, последующее колебание на более низком уровне и отдельные локальные подъемы. Наиболее трудными для прогноза остаются краткосрочные всплески — например, отдельные дни с резко повышенной выручкой. Модель лучше описывает устойчивую динамику, чем внезапные пики.

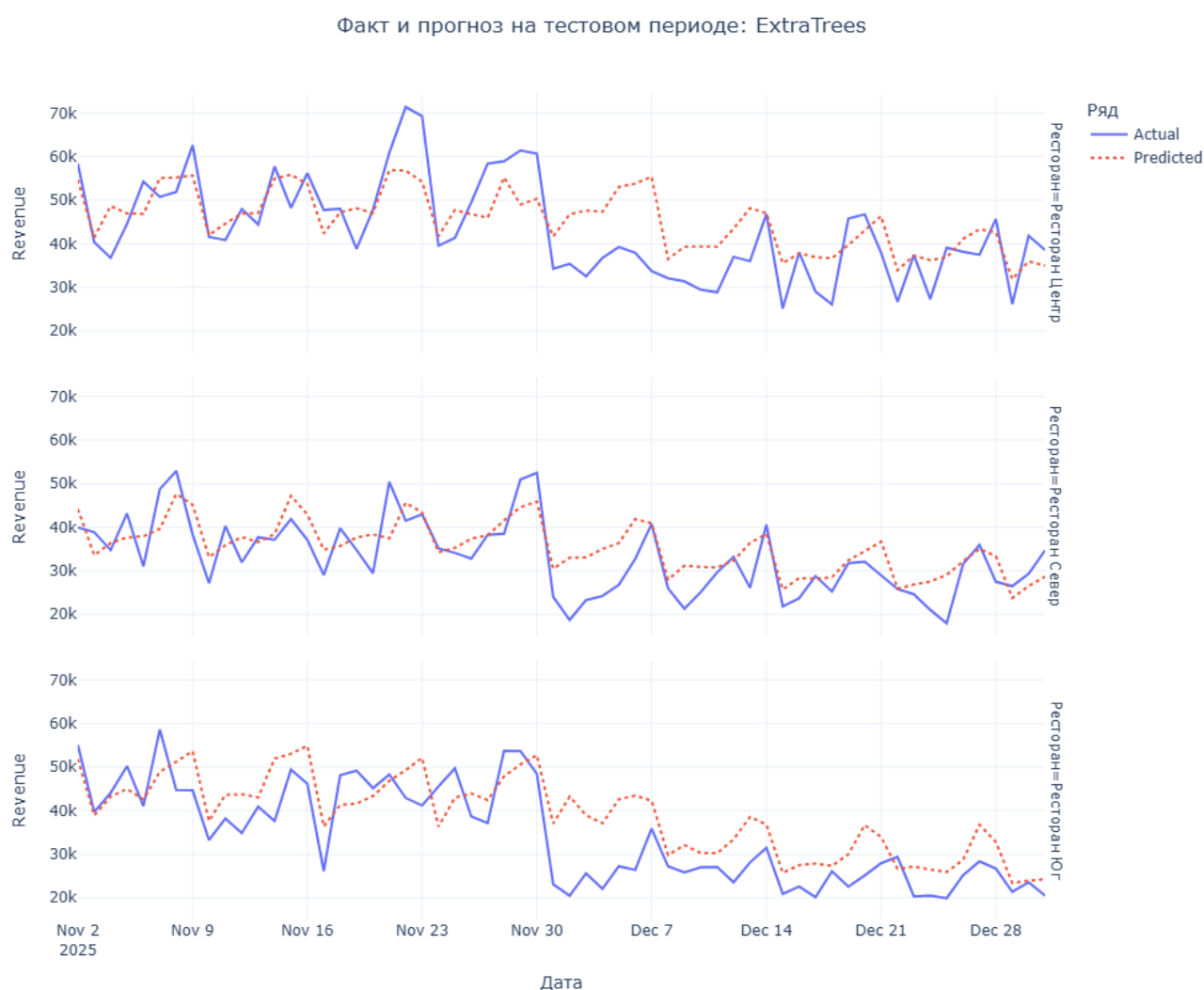


Рисунок 3.11 Сопоставление фактической и прогнозной выручки на тестовом периоде для модели ExtraTrees

Вкладка «Прогноз продаж» расширяет аналитический дашборд за счет сценарного прогноза. Пользователь работает не с кодом, а с набором управляющих элементов: выбирает целевой показатель, модель, горизонт расчета и рестораны для отображения. В интерфейсе предусмотрены четыре целевые переменные — выручка, прибыль, количество проданных блюд и число уникальных клиентов. За счет этого один экран подходит не только для финансовой оценки, но и для анализа спроса и клиентского потока. Выбор модели вынесен в отдельный список. Пользователь может запустить конкретный алгоритм — Ridge, RandomForest, ExtraTrees, HistGradientBoosting или базовую медианную модель. Горизонт задается ползунком от 7 до 90 дней. На представленном примере выбран период 14 дней, целевая переменная — выручка, модель — Extra Trees, а в фильтре оставлены все три ресторана. После изменения любого параметра дашборд автоматически пересчитывает данные и обновляет график.

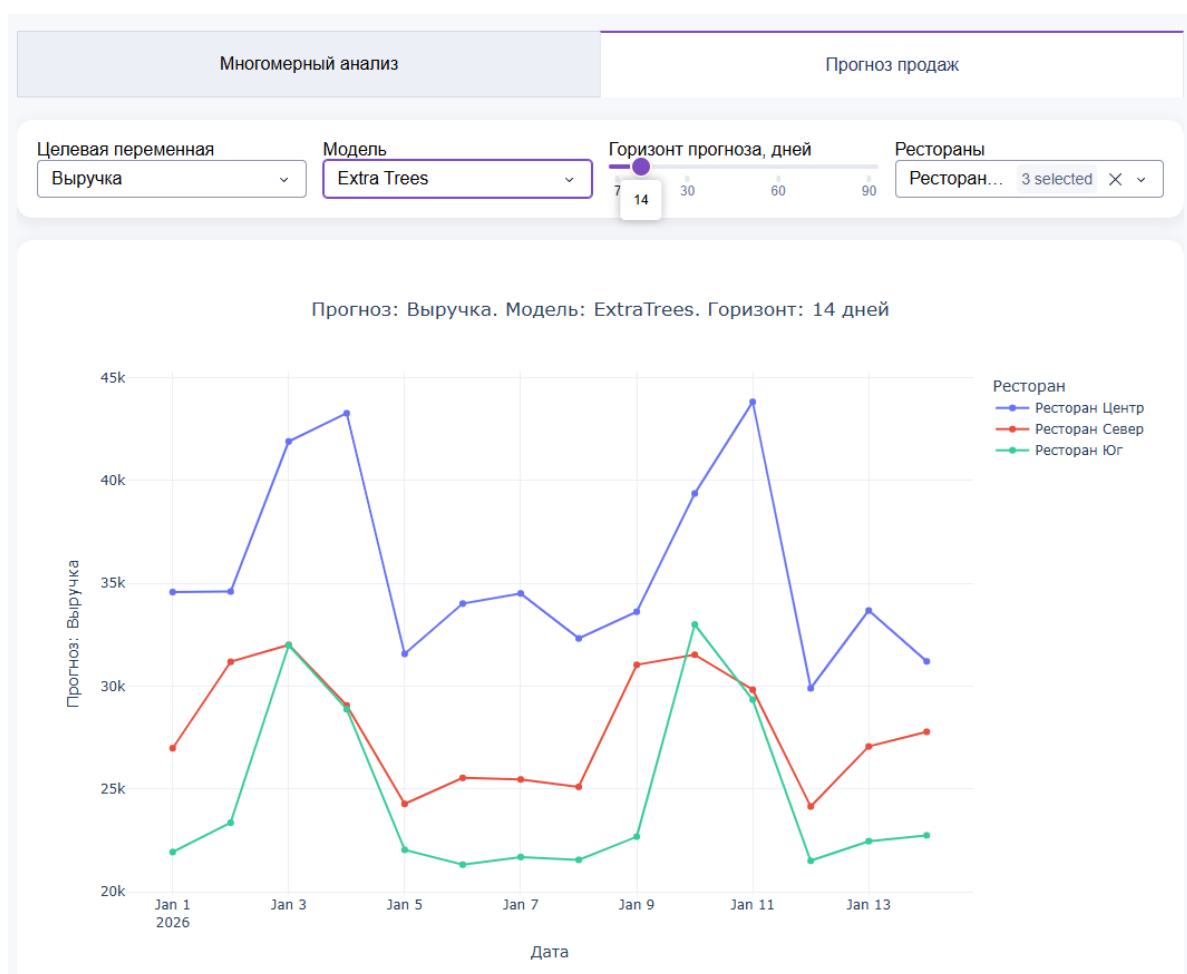


Рисунок 3.12 Модуль прогнозирования

Разработанная система полезна тем, что переводит разрозненные сведения о продажах, клиентах, ресторанах и меню в единую аналитическую среду. Руководитель или аналитик получает не набор отдельных таблиц, а инструмент, который показывает состояние бизнеса через понятные показатели: выручку, прибыль, маржинальность, количество проданных блюд, поток клиентов, средний чек и рейтинг. Для операционного управления важен прогноз спроса. Если модель показывает рост выручки или клиентского потока в ближайшие дни, ресторан может заранее увеличить закупки, усилить смену персонала и подготовить склад. Если прогноз указывает на спад, руководство может запустить акцию, сократить избыточные расходы или перенести часть ресурсов на более загруженные дни.

Заключение

В ходе выпускной квалификационной работы разработан прототип аналитической системы для ресторанного бизнеса. Система объединяет OLAP-хранилище данных, интерактивный дашборд и модуль машинного обучения, что позволяет перейти от разрозненных записей о продажах к целостному анализу деятельности ресторанной сети. Практическая часть работы включает создание хранилища данных, наполнение его демонстрационной информацией и разработку аналитических представлений. Скрипт формирования базы создаёт таблицы, индексы, календарное и временное измерения, справочники ресторанов, меню и клиентов, а затем генерирует продажи. Благодаря этому прототип можно воспроизводить повторно и использовать для проверки отчётов, дашборда и моделей прогнозирования.

Полученные результаты показывают, что разработанная система решает поставленную цель. Она формирует аналитическую среду, где пользователь может не только просматривать показатели за прошлые периоды, но и оценивать ожидаемую динамику. Для руководителя ресторана это означает более точное планирование закупок, смен персонала, маркетинговых акций и загрузки филиалов. Для аналитика система создаёт удобную основу для поиска закономерностей, проверки гипотез и дальнейшего развития моделей.

Итогом работы стал работоспособный прототип, который демонстрирует практическую ценность OLAP-хранилища и машинного обучения для ресторанной аналитики. Проект показывает, как данные о продажах, клиентах, меню и филиалах можно превратить в инструмент поддержки решений — от текущего контроля показателей до прогноза будущей нагрузки.

Список источников

1. Щербинин В. А., Степаненко В. П. Microsoft SQL Server Analysis Services. OLAP и многомерный анализ данных. — М.: ДМК Пресс, 2012. — 352 с.
2. Бергер А. Б., Кузнецов А. А., Митрофанов А. А. Microsoft SQL Server 2005 Analysis Services. OLAP и многомерный анализ данных. — СПб.: БХВ-Петербург, 2006. — 416 с.
3. Короткевич Д. SQL Server. Настройка и оптимизация для профессионалов. — СПб.: Питер, 2021. — 480 с.
4. Митин А. И. Работа с базами данных Microsoft SQL Server. Сценарии практических занятий. — М.: Директмедиа Паблишинг, 2020. — 200 с.
5. Овсяническая Л. Ю. Применение технологии OLAP-кубов для анализа данных педагогического мониторинга // Вестник БФУ им. И. Канта. — 2012. — № 2. — С. 271–279.
6. Споффорд Дж., Харинатх С. MDX Solutions: With Microsoft SQL Server Analysis Services. — Indianapolis: Wiley Publishing, 2006. — 744 p.
7. Щербинин В. А., Степаненко В. П. OLAP и многомерный анализ данных. — М.: ДМК Пресс, 2010. — 320 с.
8. Введение в MDX. КомпьютерПресс. — 2008. — [Электронный ресурс]. — Режим доступа: <https://compress.ru/article.aspx?id=9512>
9. Методы и модели анализа данных: OLAP и Data Mining. СПб.: БХВ-Петербург, 2004. — 336 с.
10. Барсегян А. А. Методы и модели анализа данных: OLAP и Data Mining: учеб. пособие по специальности 071900 "Информ. системы и технологии". — СПб.: БХВ-Петербург, 2004. — 336 с.
11. Kimball R., Ross M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. — 3rd ed. — Indianapolis: John Wiley & Sons, 2013. — 608 p.
12. Inmon W. H. Building the Data Warehouse. — 4th ed. — Indianapolis: Wiley, 2005. — 576 p.
13. Chaudhuri S., Dayal U. An Overview of Data Warehousing and OLAP Technology // ACM SIGMOD Record. — 1997. — Vol. 26, № 1. — P. 65–74.
14. Gray J., Chaudhuri S., Bosworth A., Layman A., Reichart D., Venkatrao M., Pellow F., Pirahesh H. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals // Data Mining and Knowledge Discovery. — 1997. — Vol. 1. — P. 29–53.
15. Albrecht J., Lehner W. On-Line Analytical Processing in Distributed Data Warehouses // Proceedings of the International Database Engineering and Applications Symposium. — Cardiff, 1998.
16. Akinde M. O., Böhlen M. H., Johnson T., Lakshmanan L. V. S., Srivastava D. Efficient OLAP Query Processing in Distributed Data Warehouses. — 2002.
17. Roy S. Efficient OLAP Query Processing Across Cuboids in Distributed Data Warehousing Environment // Expert Systems with Applications. — 2024.
18. Kashfi M., Hajmoosaei A. Optimal Distributed Data Warehouse System Architecture. — 2014.
19. Ramachandran R., Nair D. P., Jasmi J. A Horizontal Fragmentation Method Based on Data Semantics. — 2016.
20. Bernardino J., Madeira H. Improving Query Performance Using Distributed Computing. — Berlin: Freie Universität Berlin, 2000.

21. Han J., Kamber M., Pei J. Data Mining: Concepts and Techniques. — 3rd ed. — Waltham: Morgan Kaufmann, 2012.
22. Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. — 2nd ed. — New York: Springer, 2009.
23. Géron A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. — 3rd ed. — Sebastopol: O'Reilly Media, 2022.
24. Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45. — P. 5–32.
25. Geurts P., Ernst D., Wehenkel L. Extremely Randomized Trees // Machine Learning. — 2006. — Vol. 63. — P. 3–42.
26. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine // The Annals of Statistics. — 2001. — Vol. 29, № 5. — P. 1189–1232.
27. Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. — 3rd ed. — Melbourne: OTexts, 2021.
28. Hyndman R. J., Koehler A. B. Another Look at Measures of Forecast Accuracy // International Journal of Forecasting. — 2006. — Vol. 22, № 4. — P. 679–688.
29. Pedregosa F., Varoquaux G., Gramfort A., Michel V., Thirion B., Grisel O., Blondel M., Prettenhofer P., Weiss R., Dubourg V. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
30. McKinney W. Data Structures for Statistical Computing in Python // Proceedings of the 9th Python in Science Conference. — 2010. — P. 56–61.
31. Raasveldt M., Mühleisen H. DuckDB: An Embeddable Analytical Database // Proceedings of the 2019 International Conference on Management of Data. — 2019.
32. SQLAlchemy Documentation. SQLAlchemy 2.0 Documentation. — Электронный ресурс. — Дата обращения: 14.05.2026.
33. DuckDB Documentation. Python API. — Электронный ресурс. — Дата обращения: 14.05.2026.
34. Dash Documentation & User Guide. Plotly. — Электронный ресурс. — Дата обращения: 14.05.2026.
35. Scikit-learn User Guide. Machine Learning in Python. — Электронный ресурс. — Дата обращения: 14.05.2026.